

IMPLEMENTACIÓN Y EXTENSIÓN DE UN PROCESADOR RISCV-V SOBRE CHISEL Y FPGA

(Implementation and Extension of a RISC-V Processor
based on Chisel and FPGA)

Trabajo de Fin de Máster
para acceder al

Máster Universitario en Ingeniería Informática

Autor: Noé Ruano Gutiérrez
Director: Pablo Prieto Torralbo
Julio - 2025

TODO:

Resumen

TODO:

Abstract

TODO:

Índice general

1. Introducción	6
1.1. Motivación	6
1.2. Objetivos	6
1.3. Planificación y metodología seguidos	7
1.3.1. Formación previa	7
1.3.2. Construcción del computador monociclo	7
1.3.3. Construcción del procesador segmentado	7
1.3.4. Despliegue del diseño final sobre un FPGA	8
2. RISC-V	9
2.1. Historia de RISC-V	9
2.2. Especificaciones	10
2.3. El conjunto de instrucciones base RV32I	10
2.4. Formatos derivados	11
2.4.1. Formato B	11
2.4.2. Formato J	11
2.5. Extensiones	12
2.5.1. Extensión ‘M’	12
2.5.2. Extensión ‘Zicsr’	12
2.5.3. Extensión ‘F’	12
2.5.4. Extensión ‘Zifencei’	12
3. Herramientas y tecnologías	13
3.1. Chisel	13
3.1.1. Desarrollo en Chisel	13
3.1.2. Pruebas en Chisel	14
3.2. SBT	15
3.3. RISC-V GNU Toolchain	15
3.4. GTKWave	15
3.5. FPGA Arty A7-100T y AMD Vivado Design Suite	16
3.6. Git	17
4. El procesador monociclo	18
4.1. Descripción de la arquitectura de partida	18
4.1.1. TopModule	18
4.1.2. Memory	19
Señales de entrada	19
Señales de salida	19
4.1.3. InstructionROM y ROMLoader	20
4.1.4. CPU	20
Puertos de entrada	21

Puertos de salida	21
4.1.5. InstructionFetch	21
Señales de entrada	22
Señales de salida	22
4.1.6. InstructionDecode	22
Señales de entrada	22
Señales de salida	23
4.1.7. InstructionExecute	23
Señales de entrada	24
Señales de salida	24
4.1.8. MemoryAccess	24
Señales de entrada	24
Señales de salida	25
4.1.9. WriteBack	25
Señales de entrada	25
Señales de salida	25
5. El procesador segmentado	26
5.1. Arquitectura de un registro de etapa	28
5.2. Segmentando el núcleo en dos etapas	29
5.3. Segmentando el resto del núcleo	33
5.3.1. Resolución de riesgos de datos (RAW)	33
Riesgos causados por la sucesión de instrucciones aritmético-lógicas	34
Dependencias en las que se encuentra involucrada la instrucción 'lw'	35
5.3.2. Resolución de riesgos de control	36
6. Despliegue del diseño segmentado sobre FPGA	37
6.1. Proceso de despliegue. Ejemplo	37
6.1.1. Obtención del HDL de bajo nivel	37
6.1.2. Configuración del fichero de <i>constraints</i>	38
6.2. Modificaciones realizadas en el diseño de la CPU	39
7. Conclusiones y trabajo futuro	40

1 Introducción

1.1. Motivación

En la actualidad existen principalmente dos modelos de negocio en el ámbito del diseño y/o fabricación de procesadores. Por un lado estarían aquellas compañías cuyo beneficio proviene principalmente de la venta de licencias para el uso de sus diseños, como pudiera ser el caso de ARM, al tiempo que existen otras las cuales centran su actividad empresarial tanto en el diseño como en la fabricación y venta de hardware (casos de Intel o AMD), sin perjuicio de que pudieran obtener beneficios también del pago por el uso de su propiedad intelectual con fines comerciales [1].

Ambos modelos se basan en la siguiente premisa: que la especificación del conjunto de instrucciones que deberá ejecutar un computador constituye por sí mismo una entidad patentable. Esto obliga a todo aquel que quisiera obtener beneficio de la comercialización de productos derivados de esas especificaciones a pagar regalías al tenedor de la propiedad intelectual, lo cual obstaculiza de manera notable el surgimiento de innovaciones que pudieran aprovechar las ya existentes en materia de conjuntos de instrucciones y arquitecturas de largo recorrido como son x86-64 o ARM, cuyo éxito ha quedado demostrado históricamente.

No obstante, en los últimos años del siglo XX y principios del XXI surgieron corrientes que apostaban por la creación de arquitecturas libres con proyectos como Sparc u OpenRISC, este último aún en desarrollo [2]. Pero no sería hasta el año 2011 [3] cuando en la Universidad de Berkeley comenzaría a gestarse el que es sin lugar a dudas el ISA abierto que mayor renombre ha conseguido tomar hasta la fecha: RISC-V. Tanto es así que gigantes tecnológicos como Google, Intel, Microsoft o Nvidia [4] realizan importantes aportaciones monetarias de manera regular a la fundación que mantiene y desarrolla la especificación (RISC-V International). Y es que los beneficios del hardware y software libres son innegables, puesto que facilitan la educación y la investigación al permitir experimentar con arquitecturas personalizadas sin restricciones comerciales, acelerando la creación de prototipos y contribuyendo a democratizar el acceso a la tecnología.

Además de compañías privadas, multitud de instituciones públicas contribuyen, por supuesto, a impulsar el desarrollo de la especificación. En el ámbito europeo, la EPI¹ es la iniciativa mediante la cual se canalizan los esfuerzos de la academia e industria europeas por hacer realidad el proyecto de una arquitectura abierta, robusta y capaz, que sitúe a la Unión Europea a la vanguardia de la computación a exaescala. Y es este uno de los principales motivos que han impulsado el desarrollo de este trabajo: la posibilidad de conocer en profundidad y trabajar con una de las tecnologías clave para el futuro tecnológico de la Unión Europea.

1.2. Objetivos

El principal objetivo del proyecto es la implementación de un computador sencillo, mononúcleo, segmentado y en orden, que pueda ejecutar el conjunto de instrucciones definido en la especificación del ISA abierto RISC-V, en su versión de 32 bits. Para ello se partirá de un diseño ya existente, monociclo e incompleto, elaborado por el investigador de la Universidad Nacional Cheng Kung, Ching-Chun

¹European Processor Initiative

(Jim) Huang, quien lo emplea como material docente en una asignatura de arquitectura y tecnología de computadores impartida en esa misma universidad.

El diseño de partida se encuentra descrito en el lenguaje Chisel, que es un lenguaje de propósito específico o DSL (Domain-Specific Language) construido sobre el lenguaje de programación Scala, este último, un lenguaje de propósito general o GPL (General Purpose Language). Esto requerirá la adquisición, por parte del autor del presente trabajo, del conocimiento sobre ambos lenguajes que le permita, a posteriori, afrontar la tarea de completar el pipeline del mencionado diseño original para, más adelante, modificarlo de manera que este se encuentre segmentado en cinco etapas.

La segmentación del diseño original implicará, en primer lugar, un estudio profundo de la estructura interna del procesador, analizando el comportamiento y el conexionado de los diferentes bloques en los que se encuentra organizada la arquitectura del núcleo para, más adelante, integrar en él los registros de etapa y la unidad de riegos con los que se transformará la arquitectura monocíclica en una arquitectura en pipeline.

Durante todo el proceso se hará uso de las herramientas descritas más adelante en el Capítulo 3, las cuales posibilitarán la identificación de los errores cometidos en el proceso de segmentación de la arquitectura.

Por último, se pretende elaborar una prueba de concepto consistente en el despliegue del diseño final sobre una placa FPGA (Field Programmable Gate Array), de modo que pueda comprobarse sobre hardware real el buen funcionamiento y desempeño de este diseño.

1.3. Planificación y metodología seguidos

1.3.1. Formación previa

En primer lugar se realizará una formación relativa a los lenguajes Scala y Chisel². Esta formación fue creada por diversos investigadores y colaboradores de la Universidad de Berkeley y consiste en un cuaderno de Jupyter en el que se abordan, inicialmente, aspectos introductorios relativos al desarrollo de software en el lenguaje Scala para, posteriormente, proporcionar al usuario nociones fundamentales sobre el diseño de hardware empleando el lenguaje Chisel en lecciones de complejidad incremental.

1.3.2. Construcción del computador monociclo

La segunda fase del proyecto consistirá en la implementación de las partes incompletas en la arquitectura de partida. Ello implicará la puesta en práctica y consiguiente asentamiento de los conocimientos adquiridos durante la realización del bootcamp sobre Scala y Chisel, además de que constituirá en sí mismo un proceso de descubrimiento de la arquitectura del núcleo monociclo y las particularidades de su implementación.

1.3.3. Construcción del procesador segmentado

A continuación, con el diseño base completado, se procederá a segmentar el núcleo en las cinco etapas siguientes:

1. Lectura de instrucción o fase de *fetch*
2. Decodificación de la instrucción
3. Ejecución
4. Lectura/escritura de resultados en memoria
5. Escritura en el banco de registros, o fase de *writeback*

²Disponible en: www.github.com/freechipsproject/chisel-bootcamp

La integración de los registros de etapa requerirá un análisis minucioso del conexionado interno del procesador, e implicará la alternancia continua de fases de desarrollo (construcción y adhesión de los registros al pipeline) y fases de prueba hasta verificar que la ejecución de los binarios cargados desemboca en los resultados esperados a nivel del estado arquitectónico.

1.3.4. Despliegue del diseño final sobre un FPGA

Para concluir el proyecto se elaboró una prueba de concepto consistente en el despliegue del diseño elaborado sobre una placa FPGA diseñando, también en lenguaje Chisel, una interfaz con la que poder comunicar el computador con los dispositivos presentes en la placa. Ello posibilitaría el envío, carga en memoria y ejecución de binarios compilados para la arquitectura RISC-V, gracias a un módulo UART desarrollado por el investigador del grupo de Ingeniería de Computadores y director de este trabajo, Pablo Prieto Torralbo.

2 RISC-V

RISC son las siglas de ‘Reduced Instruction Set Computer’, es decir, un computador con un conjunto de instrucciones reducido. Una de las primeras menciones de este concepto la encontramos en la década de 1970. En aquel entonces, en IBM, tenían la necesidad de desarrollar un computador sencillo, pero capaz de ejecutar 12 millones de instrucciones por segundo, de modo que pudiera ser integrado en un sistema de enrutamiento de llamadas telefónicas [5].

En ese periodo muchas arquitecturas comerciales seguían el paradigma CISC (Complex Instruction Set Computer), caracterizado por conjuntos de instrucciones amplios y relativamente complejos. Con frecuencia, esas instrucciones deben ser interpretadas a nivel arquitectural y traducidas a un microcódigo con el que se ejecutaban las operaciones necesarias para obtener los resultados esperados. No obstante, este enfoque requería contar, en primer lugar, con un intérprete de bajo nivel que tradujera las instrucciones en sus correspondientes microcódigos, además de una memoria dedicada en la que alojarlos.

Las arquitecturas RISC surgen como respuesta a la tendencia de los conjuntos de instrucciones a ganar complejidad con el tiempo, a pesar de que algunos estudios [6] apuntaban que la mayoría de los programas desarrollados por aquel entonces tan solo empleaban un subconjunto de las instrucciones disponibles. En estos estudios se proponía la creación de arquitecturas más sencillas, baratas de producir y eficientes, las cuales, aún siéndolo, fueran capaces de ejecutar programas complejos de igual modo que lo hacían las arquitecturas CISC.

Con el tiempo, numerosos principios asociados al diseño RISC han sido adoptados de manera generalizada en las arquitecturas de procesadores modernos. De hecho, incluso aquellas tradicionalmente clasificadas como CISC emplean internamente representaciones simplificadas de sus instrucciones, lo que facilita su ejecución en pipelines profundos y en arquitecturas con ejecución fuera de orden. En consecuencia, la distinción clásica entre RISC y CISC ha tendido a difuminarse.

En los últimos años, el interés por arquitecturas basadas en estos principios se ha intensificado con la aparición de arquitecturas abiertas que permiten estudiar, modificar e implementar procesadores completos sin las limitaciones impuestas por especificaciones propietarias. En este contexto, RISC-V se fundamenta en los principios clásicos del diseño RISC, caracterizados por la simplicidad del conjunto de instrucciones, la regularidad de su formato y la eficiencia en la ejecución. A estos elementos añade un modelo de especificación abierto que permite su implementación, modificación y extensión sin costes de licencia ni restricciones adicionales, facilitando la experimentación y la personalización de la arquitectura y fomentando la colaboración entre los ámbitos académico e industrial. Como resultado, la combinación de simplicidad arquitectónica y accesibilidad ha impulsado una rápida adopción de RISC-V en diversos contextos, consolidándola como una plataforma especialmente idónea tanto para la docencia como para la investigación en arquitectura de computadores.

2.1. Historia de RISC-V

El manual de la primera versión del ISA abierto RISC-V fue publicado en el año 2011. Por aquel entonces, los investigadores de la Universidad de California en Berkeley, Krste Asanović, Yunsup Lee y Andrew Waterman, bajo la dirección de David A. Patterson, integraban el equipo *ParLab*, trabajando en un proyecto de investigación sobre computación paralela, financiado por Intel y Microsoft, y del cual surgieron como derivados del trabajo de investigación tanto el ISA RISC-V, como el lenguaje de

construcción de hardware (HDL) Chisel [7].

Más tarde, en el año 2015, se creó la RISC-V Foundation con el objetivo de organizar los esfuerzos de la comunidad global de desarrolladores de software y hardware, por impulsar la adopción del estándar, mantenerlo y hacerlo evolucionar de una manera organizada y eficaz. Además, en este mismo año, los integrantes del *ParLab* (a excepción de Patterson) fundarían SiFive, una compañía dedicada al diseño y venta (que no fabricación) de procesadores basados en RISC-V.

2.2. Especificaciones

La especificación la componen dos ISAs, uno estándar y otro privilegiado, siendo el primero aquel sobre el que se ha puesto el foco a la hora de proponer y desarrollar este trabajo. La especificación del ISA privilegiado recoge las instrucciones y modos de ejecución necesarios para ejecutar sistemas operativos y operar con periféricos [8], mientras que la especificación del ISA no privilegiado recoge las instrucciones necesarias para la construcción de arquitecturas RISC-V básicas pero completamente funcionales.

El ISA no privilegiado lo componen, a su vez, cuatro especificaciones básicas junto con sus respectivas extensiones. Esas especificaciones básicas deben estar presentes en toda implementación que quiera hacerse del ISA, es decir, que una implementación debería integrar una de las cuatro especificaciones base, junto con tantas extensiones como se requiera. Las especificaciones base recogen el conjunto de instrucciones básico que todo núcleo RISC-V debería ser capaz de ejecutar [9], y son las siguientes:

- **RV{32,64}I**: ISAs con un XLEN (longitud de registro) de 32 y 64 bits.
- **RV{32,64}E**: subconjuntos de RV32I y RV64I con la mitad de registros. Elaborados para propiciar la construcción de microcontroladores de tamaño reducido.

Las instrucciones de las especificaciones base posibilitan la ejecución de operaciones aritmético-lógicas (suma, resta, operaciones lógicas, *lui*¹ y *auipc*²), saltos condicionales y no condicionales, así como operaciones de lectura y escritura en memoria [10]. En el ISA base se definen, además, las operaciones de ordenación de memoria necesarias para garantizar la consistencia en sistemas con múltiples hilos de ejecución paralelos o (*harts*) [11].

Asimismo, se definen las instrucciones ‘ECALL’ y ‘EBREAK’ cuya utilidad es, respectivamente, realizar llamadas al sistema y pausar la ejecución de un *hart* para inspeccionar el estado del banco de registros y la memoria (útil en labores de depuración) [12].

Por último, la especificación indica el formato de otro tipo de instrucciones las cuales, aún siendo válidas, pueden no tener ningún efecto sobre el estado de la arquitectura. Estas instrucciones, denominadas ‘HINTs’, se reservan para casos en que se desee agregar una cierta funcionalidad al ISA, la cual no tenga por qué ser adoptada obligatoriamente por una arquitectura ya existente, pudiendo esta simplemente ignorar la instrucción [13].

2.3. El conjunto de instrucciones base RV32I

En el manual de la arquitectura se indica que en el conjunto RV32I las instrucciones deben poder direccionar a 32 registros de propósito general. No obstante, tan solo se restringe el uso del registro ‘x0’ para albergar una constante de valor 0, quedando recogido el uso del resto de registros en la ABI³ de RISC-V [14].

En lo que respecta al formato de las instrucciones, existen cuatro formatos base [15]:

¹*load upper immediate*: carga en un registro un entero de 32 bits construido con los 20 bits más significativos de la instrucción (inmediato), a los cuales se aplica un shift lógico de 12 bits hacia la izda.

²*add upper immediate (to) program counter*: construye un inmediato de 32 bits exactamente igual que *lui*, lo suma al valor del contador de programa y lo guarda en un registro

³*Application Binary Interface*: una especificación de la forma en que los programas deben ser llamados a nivel de lenguaje máquina (paso de parámetros, retorno de resultados, etc.), así como la forma en que deben construirse y enlazarse los ejecutables que los albergan

- **R**: operaciones aritmético-lógicas en las que los dos operandos son registros.
- **I**: operaciones aritmético-lógicas en las que uno de los operandos es un inmediato codificado en la propia instrucción. Este formato es el empleado, además, en la codificación de la instrucción de lectura de memoria *lw* y sus derivados, así como en la codificación de la instrucción de salto incondicional *jalr*⁴.
- **S**: instrucción de escritura en memoria *sw* y sus derivados.
- **U**: es el formato empleado en la codificación de las instrucciones *lui* y *auipc*.

2.4. Formatos derivados

2.4.1. Formato B

El formato B es una variación del S y se emplea en la codificación de las instrucciones de salto condicional. La diferencia entre este formato y el formato S reside en la manera de interpretar los campos con que se construye el inmediato.

En las instrucciones de escritura en memoria, la dirección efectiva de acceso se forma concatenando las dos secciones del inmediato (la más significativa y la menos significativa) dentro de la instrucción, extendiendo el signo del resultado a 32 bits, y sumándolo al contenido del registro *rs1*. Por otro lado, en la decodificación de las instrucciones de salto condicional, el offset respecto del contador de programa se construye de una manera menos intuitiva, tal y como puede verse en la Figura 2.1.

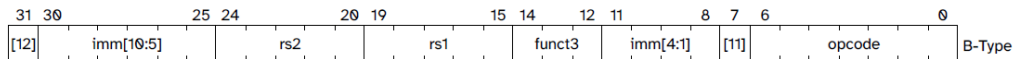


Figura 2.1: Esquema con el formato que deben seguir las instrucciones del tipo B

El offset siempre va a ser un múltiplo de 2, por lo que el último dígito binario del inmediato puede asumirse siempre nulo. Con esto, el inmediato en las instrucciones de salto condicional podría codificar los 12 bits más significativos siguientes a ese primer dígito binario, quedando un offset de 13 bits que, posteriormente, habría que extender a 32 bits para sumarlo al contador de programa. En aras de favorecer la reutilización de la lógica de decodificación del inmediato del formato S, se decidió construir el formato B de forma que hubiera tanta superposición como fuera posible con el S, facilitando de este modo la decodificación. En resumidas cuentas, se sacrificó ligeramente la legibilidad del inmediato dentro del formato, favoreciendo la eficiencia en la implementación.

2.4.2. Formato J

El formato J debe emplearse en la codificación de las instrucciones de salto incondicional. Se trata de una variación del tipo U, que sigue la lógica del formato B en cuanto a la construcción del inmediato, tal y como puede observarse en la Figura 2.2 el offset se encuentra codificado en múltiplos de 2 por lo que el último bit va a ser siempre 0, con lo que puede codificarse dentro de la instrucción los 19 bits más significativos inmediatamente posteriores a ese último dígito binario.

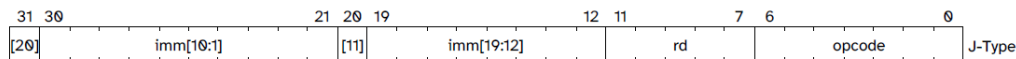


Figura 2.2: Esquema con el formato que deben seguir las instrucciones del tipo J

⁴*jump and link register*: avanza el contador de programa a una posición igual a su valor actual más el offset formado con la extensión de signo del inmediato a 32 bits. Además, guarda la dirección de retorno (instrucción siguiente al salto) en un registro de destino codificado en la instrucción

2.5. Extensiones

A continuación se presentan algunas de las principales extensiones contempladas en la especificación de RISC-V, diseñadas para ampliar las funcionalidades del conjunto base de instrucciones y permitir su adaptación a diferentes dominios de aplicación.

2.5.1. Extensión ‘M’

Para simplificar la implementación del conjunto de instrucciones base se decidió incorporar las instrucciones de multiplicación y división de enteros a su propia extensión en la que se definen, de manera separada, el formato de las instrucciones de multiplicación (MUL, MULHU y MULHSU⁵) y el de las instrucciones de división (DIV⁶, DIVU⁷ y REM(U)⁸).

2.5.2. Extensión ‘Zicsr’

La especificación del conjunto de instrucciones base no contempla la presencia de un registro de estado y control o ‘CSR’⁹. No obstante a ello, en la especificación del ISA no privilegiado sí se definen un conjunto de operaciones que son consideradas de utilidad en sistemas que gestionen múltiples *harts*. Estas operaciones incluyen el manejo de contadores y temporizadores, así como la notificación del estado de las operaciones de punto flotante [16].

2.5.3. Extensión ‘F’

La extensión ‘F’ recoge las operaciones de punto flotante en precisión simple, y define sus formatos y comportamiento, así como la presencia de un conjunto de 32 registros dedicados, además de un registro de estado propio (*fcsr*) y la disposición de sus campos.

2.5.4. Extensión ‘Zifencei’

En esta extensión se detallan el formato y comportamiento de la instrucción *FENCE.I*, empleada en la sincronización entre escrituras sobre la memoria de instrucciones, y las lecturas sobre esa misma memoria [17]. Esta instrucción toma especial relevancia en contextos donde el código de un *hart* que ejecuta en un sistema con arquitectura Harvard¹⁰, puede hacer que se sobrescriba a sí mismo. Tras una escritura en un área de la memoria correspondiente al código del programa, este podría invocar la instrucción *FENCE.I* para garantizar la coherencia en la memoria de instrucciones con respecto a la de datos, que es donde realiza la escritura. La primera, en el momento de ejecutar la instrucción escrita previamente, podría leer en la dirección de la instrucción un dato inconsistente, en función de si alguna lectura de instrucciones anteriores hubiera forzado o no la actualización de esa posición en la memoria.

⁵Las instrucciones MULHU y MULHSU retornan los *XLEN* bits más significativos en resultados cuya representación requiera de, a lo sumo, $2 * XLEN$ bits

⁶Cociente con signo

⁷Cociente sin signo

⁸Resto de los cocientes con y sin signo

⁹Control-Status Register

¹⁰Arquitecturas donde la memoria se encuentra dividida, proporcionando buses dedicados e independientes para acceder a la memoria de datos y a la de instrucciones

3 Herramientas y tecnologías

3.1. Chisel

Chisel es un lenguaje de descripción de hardware de alto nivel gestado en el año 2012 en el seno del Departamento de Ingeniería Eléctrica y Ciencias de la Computación (abrev. EECS) de la Universidad de California, en Berkeley, y surgió como parte de un proyecto de investigación cuyo objetivo final era la implementación de una interfaz de lenguaje con la que elevar el nivel de abstracción ofrecido por HDL's convencionales como son VHDL o Verilog, ofreciendo la posibilidad de traducir los desarrollos elaborados, en entidades descritas en estos mismos lenguajes de bajo nivel, con objeto de correr procesos de síntesis o emulación que permitiesen evaluar su funcionamiento [18].

El hecho de que se trate de un HDL de alto nivel tiene un impacto directo y positivo sobre la celeridad en los procesos de desarrollo y prueba del hardware, permitiendo elaborar y probar diseños de una manera ágil sin perder las características de eficiencia y robustez que proporcionan lenguajes de más bajo nivel.

Es un lenguaje de propósito específico construido sobre Scala, un lenguaje de propósito general orientado a objetos, siendo esta la principal propiedad que permite a Chisel mostrar al usuario una interfaz de diseño de hardware de tan alto nivel y, por ende, amigable tanto para el desarrollador novel como para usuarios más avanzados, y es que resulta muy práctico el poder modelar un circuito lógico como un conjunto de clases relacionadas entre sí. Cada una de estas clases define el comportamiento interno de un componente específico, mientras que las relaciones entre las distintas clases que conforman el diseño global representan las conexiones físicas entre los módulos del sistema. Este enfoque no solo facilita la estructuración del diseño, sino que también permite trasladar al desarrollo hardware principios propios de la ingeniería del software tales como la abstracción, la modularidad y la reutilización de código, favoreciendo así la construcción de sistemas complejos a partir de componentes bien definidos y fácilmente mantenibles.

3.1.1. Desarrollo en Chisel

La unidad fundamental de diseño de hardware en Chisel son los módulos, que encapsulan la disposición interna y el comportamiento de un circuito lógico. Esta disposición interna pueden componerla otros módulos, cables internos o cables de entrada/salida, cuyos valores serán empleados en la evaluación de las expresiones lógicas que definan el comportamiento del módulo, y que pueden describirse por medio de operadores aritmético-lógicos convencionales, o bien, por medio de los tipos de datos y componentes definidos en las librerías de Chisel, tales como multiplexores, decodificadores o incluso memorias.

A continuación, en la Figura 3.1, se proporciona un código sencillo con el que se describe un multiplexor de 4 entradas de 32 bits cada una. Nótese que los módulos se definen como clases que extienden la clase *Module*, lo cual obliga a contar con, al menos, una interfaz encapsulada en el método *IO*, por medio del cual se definirá esa interfaz como un elemento de entrada/salida. En el caso del código mostrado en la Figura 3.1 las interfaces de entrada/salida se encuentran definidas en las líneas 7 a 14 como parte de una instancia de la clase *Bundle*, que es un constructo que permite en Chisel crear nuevos tipos de datos al estilo de los *struct* en el lenguaje C.

```

1 package mux4
2 import chisel3._
3 import chisel3.util._
4
5 class Mux4 extends Module {
6   val io = IO(new Bundle{
7     val i1 = Input(UInt(32.W))
8     val i2 = Input(UInt(32.W))
9     val i3 = Input(UInt(32.W))
10    val i4 = Input(UInt(32.W))
11    val sel = Input(UInt(2.W))
12
13    val o = Output(UInt(32.W))
14  })
15
16   io.o := MuxLookup(io.sel, io.i1)(
17     Seq(
18       0.U -> io.i1,
19       1.U -> io.i2,
20       2.U -> io.i3,
21       3.U -> io.i4,
22     ))
23 }

```

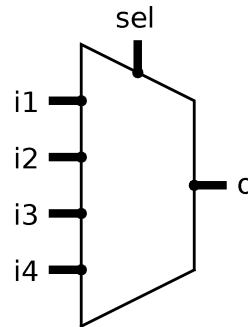


Figura 3.1: Código y diagrama del multiplexor de 4 entradas en Chisel

3.1.2. Pruebas en Chisel

Como se comentaba al comienzo de esta sección sobre Chisel, una de sus características más reseñables es la agilidad con la que puede probarse el comportamiento de los módulos, y ello es gracias a los tests, los cuales son definiciones de clases que implementan uno de los *traits* de Chisel en los que se definen a su vez los procedimientos que permiten evaluar el funcionamiento de los módulos. Estos *traits* son interfaces que definen una serie de funciones básicas con las que comprobar y alterar el estado de las señales dentro de los módulos.

Se ha decidido utilizar *ChiselScalatestTester* como interfaz de pruebas en lugar de la más reciente *ChiselSim*. Dado que no se aprecian diferencias relevantes a nivel de implementación entre ambas interfaces, la elección se ha fundamentado en la equivalencia funcional entre ellas y en la necesidad de reutilizar el conjunto de tests ya desarrollados, evitando así esfuerzos de adaptación innecesarios.

Las interfaces de prueba de Chisel definen cuatro funciones básicas con las que evaluar el comportamiento de un módulo: *peek*, *poke*, *expect* y *step*. Estas funciones se invocan sobre las diferentes señales de las cuales se componga el módulo y permiten realizar las siguientes acciones:

- ***peek***: lectura del valor de una señal en el instante de invocación de la función.
- ***poke***: cambio en el valor de la señal.
- ***expect***: evaluación de un valor determinado en la señal, esto es, probar si el valor de la señal en el instante de invocación de la función es el esperado.
- ***step***: se invoca sobre una señal de reloj y permite avanzarlo un ciclo.

En la Figura 3.2 se muestra un ejemplo de test que evalúa el comportamiento del módulo definido en el código de la Figura 3.1. En el código del test se establece el valor de los cuatro puertos de entrada por medio del método *poke* y, luego, se itera con los posibles valores de la señal de selección, comprobando, por medio del método *expect*, que el valor en el puerto de salida es el esperado.

```

1 package mux4
2 import chisel3._
3 import chiseltest._
4 import org.scalatest.flatspec.AnyFlatSpec
5 import mux4.Mux4
6
7 class Mux4Test extends AnyFlatSpec with ChiselScalatestTester {
8   behavior.of("Mux4")
9   it should "output the right input, given any valid selector value" in {
10     test(new Mux4) { c =>
11       c.io.i1.poke(1.U)
12       c.io.i2.poke(2.U)
13       c.io.i3.poke(3.U)
14       c.io.i4.poke(4.U)
15       var x = 0
16       for (x <- 0 to 3) {
17         c.io.sel.poke(x.U)
18         c.io.o.expect(x + 1)
19       }
20     }
21   }
22 }

```

Figura 3.2: Implementación de un test para el multiplexor de 4 entradas

3.2. SBT

SBT es una herramienta de construcción de software semejante a Maven o Gradle, y es la más utilizada por los desarrolladores en lenguaje Scala[19]. Su uso es por medio de una interfaz de línea de comandos, proporcionando una consola interactiva desde la que pueden realizarse todo tipo de tareas administrativas sobre los proyectos, tales como lanzar pruebas, ejecutar un programa principal, compilar ficheros de código fuente de manera individual o construir el proyecto, requiriendo esto último la creación de un fichero de configuración *build.sbt* dentro del directorio del proyecto.

Escrito también en el lenguaje Scala, en él se definirán las propiedades del proyecto, como puedan ser su nombre y versión, la versión de Scala empleada en la implementación o las librerías de las que depende, siendo estas dos últimas propiedades imprescindibles de cara al proceso de compilación.

3.3. RISC-V GNU Toolchain

Gran parte de las pruebas llevadas a cabo sobre la implementación del núcleo han consistido en la carga, ejecución y depuración de programas desarrollados en lenguaje C, lo cual ha implicado aplicar un proceso de compilación cruzada para obtener binarios que pudieran ejecutar sobre la arquitectura RISC-V.

Este proceso ha sido llevado a cabo por medio de la suite de herramientas de que se compone el toolchain de GNU para RISC-V[20], disponible en sistemas Linux basados en Debian por medio del gestor de paquetes *apt* (paquete *gcc-riscv64-unknown-elf*).

3.4. GTKWave

La depuración de componentes arquitecturales individuales tales como la ALU o el banco de registros, o incluso fases completas dentro del pipeline, es una tarea que puede acometerse de manera ágil por medio de tests, estableciendo primeramente los casos de prueba a implementar, y verificando que los cambios

en las señales bajo supervisión son los esperados para el conjunto de entradas definido en cada caso de prueba. No obstante, este proceso de depuración del pipeline se torna más complejo cuando se desea verificar el comportamiento global de la arquitectura mediante la ejecución de un programa, lo cual requiere la implementación de tests que verifiquen el estado de un número mucho mayor de señales a cada ciclo, incrementando notablemente la complejidad de los tests desarrollados y del proceso de depuración.

Aún siendo esta una tarea ciertamente abordable, se optó por explorar alternativas que permitiesen depurar el estado de la arquitectura de una manera más rápida y visual, y se consideró que la herramienta GTKWave cumpliría perfectamente con este propósito.

GTKWave es una herramienta de análisis empleada en la depuración de trazas generadas en los procesos de simulación de modelos elaborados con Verilog o VHDL. Estas trazas, generadas con la ejecución de los tests de Chisel, se almacenan en ficheros con un formato definido originalmente en el estándar IEEE 1364-1995[21] y constituyen una descripción de todos los cambios experimentados por las señales de los módulos durante la ejecución de las simulaciones.

La herramienta permite explorar esos cambios de una manera visual por medio de una interfaz gráfica en la que puede explorarse el histórico de cambios de la totalidad de las señales que conformen cada módulo, tal y como se muestra en la Figura 3.3, donde se presenta una captura de pantalla de la interfaz gráfica de GTKWave, mostrando una visualización de la traza generada tras la ejecución del test mostrado en la Figura 3.2.

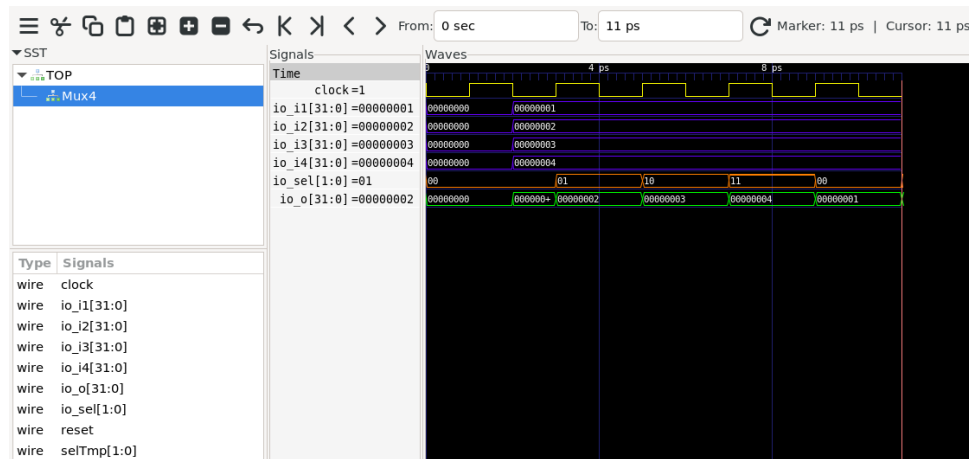


Figura 3.3: Visualización de la traza generada tras la simulación del test

3.5. FPGA Arty A7-100T y AMD Vivado Design Suite

AMD Vivado es un software de diseño hardware para SoC¹ y FPGAs desarrollado y mantenido por la empresa AMD que es, además, el fabricante de la placa de desarrollo Artit A7-100T empleada en las pruebas llevadas a cabo como parte del proceso de verificación final de la arquitectura diseñada.

La placa alberga el FPGA AMD (anteriormente Xilinx) Artix7, con 101440 celdas lógicas[22], superficie más que suficiente para albergar el núcleo segmentado. Además, tal y como puede verse en la imagen de la placa presentada en la Figura 3.5², esta cuenta con numerosos elementos de entrada/salida tales como switches de pulsación (10) y lineales (9), diodos LED (números 7 y 8), interfaces Ethernet

¹System-on-Chip: circuitos integrados que incorporan todos o la gran mayoría de los componentes que constituyen un computador

²Diligent. (2023). Arty A7 - Diligent Reference [png]. Arty A7 Reference Manual. Disponible aquí

(3) y UART (bajo USB; número 2), así como conectores para módulos de expansión de Diligent (*pmod*³; número 16) y shields de Arduino (11).

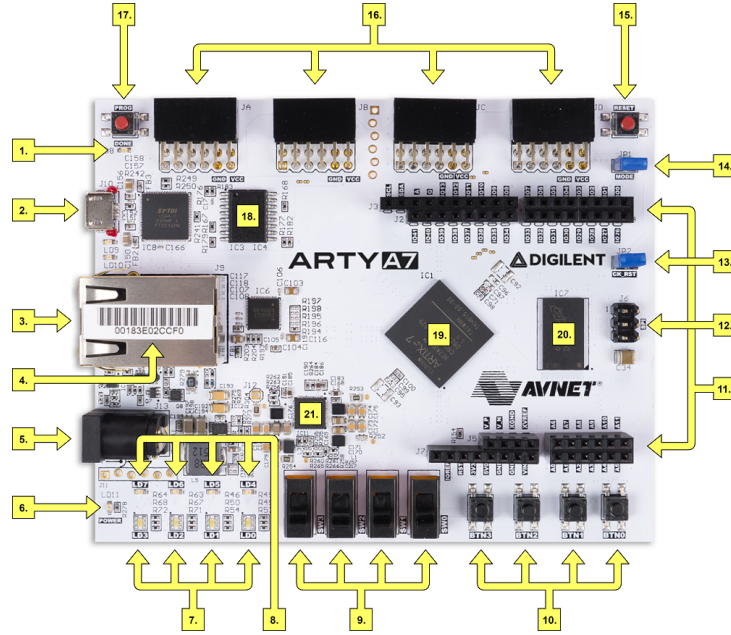


Figura 3.4: Placa de desarrollo Artix A7-100T

3.6. Git

Git fue concebido originalmente por Linus Torvalds como una alternativa de código abierto a BitKeeper, el software de control de versiones propietario empleado en el desarrollo del kernel de Linux entre los años 2002 y 2005[23], y es actualmente el software de control de versiones más utilizado por los desarrolladores, según los datos reflejados en el *Stack Overflow Annual Developer Survey* del año 2022[24]

³Peripheral module interface

4 El procesador monociclo

4.1. Descripción de la arquitectura de partida

El código que describe la composición y funcionamiento internos del pipeline original se encuentra distribuido en cinco módulos, cada uno de los cuales modela una de las cinco etapas en las que se ha decidido segmentarlo. Además, existe un módulo superior dentro del cual se establece el conexionado entre las diferentes etapas y, por encima de este, un módulo en un nivel adicional en el que se definen las conexiones entre el agregado de las unidades funcionales del pipeline y la memoria, unificada, la cual alberga datos e instrucciones.

Se proporciona a continuación, en la Figura 4.1, un diagrama con el que se espera poder ayudar a comprender las relaciones entre los distintos componentes que conforman el computador.

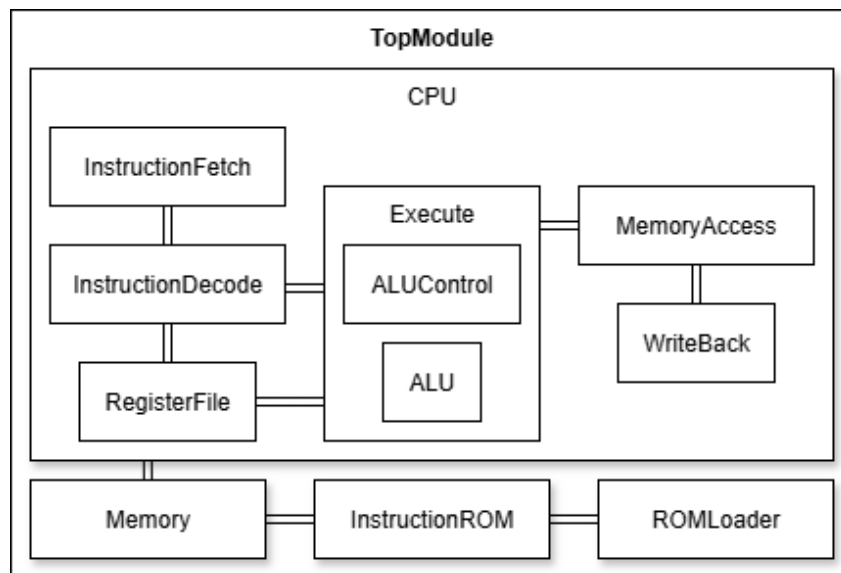


Figura 4.1: Diagrama de bloques de la arquitectura global del computador

A continuación se indicarán los módulos que conforman el diseño de partida junto con una descripción tanto de la funcionalidad que modelan, como de las señales de entrada y salida que reciben y generan respectivamente. Además, cabe indicar que los ficheros de código fuente donde se encuentra definido cada módulo siguen la siguiente nomenclatura: *«NombreModulo».scala*.

4.1.1. TopModule

TopModule es el módulo en el que se describe el conexionado de los grandes bloques que conforman el computador, es decir, el núcleo, la memoria, el cargador y la ROM de instrucciones, cuyo funcionamiento

interno e interoperabilidad serán descritos en los subapartados siguientes a este.

El módulo cuenta con una interfaz de entrada-salida cuyos puertos permiten observar el estado del banco de registros y de la memoria en cualquier momento, lo cual resulta de gran utilidad cuando se necesita depurar la ejecución de programas complejos que puedan sacar a la luz defectos en el diseño del pipeline, los cuales no hubieran podido ser detectados por medio de los tests convencionales que prueban el comportamiento individual de cada uno de los componentes que conforman el diseño global.

4.1.2. Memory

Tal y como se indicaba al comienzo de este capítulo sobre la arquitectura interna del computador, la memoria se encuentra unificada, albergando tanto instrucciones como datos, lo cual contribuye a simplificar el diseño global, haciendo posible que todas las operaciones sobre la memoria sean realizadas por medio de una misma interfaz, definida dentro de un único módulo (*Memory*) que cuenta, además, con una instancia definida dentro del módulo *TopModule*.

Sobre el código del módulo *Memory*, en él se define la clase *Memory* con la que se construyen tanto la memoria unificada del computador, como la interfaz a la que se conecta el módulo *CPU* para habilitar la comunicación entre la memoria y los módulos que componen el núcleo, más concretamente, *InstructionFetch* y *MemoryAccess*, que emplean los puertos definidos en la interfaz de la memoria para leer de manera independiente, y en paralelo, los datos e instrucciones que en ella se encuentran alojados.

A continuación se detallará el propósito de los puertos de entrada-salida del módulo. No obstante, dentro de la entrada-salida se define también un *bundle* como una instancia de la clase *RAMBundle*, definida dentro del mismo fichero donde se especifica la clase *Memory*. Al instanciar esta clase se crea un *Bundle* de Chisel, es decir, una agrupación de señales de entrada-salida, en este caso, relacionadas con la interacción con la memoria.

Señales de entrada

- ***instruction_address***: es un bus de 32 bits por donde la memoria recibe la dirección de la siguiente instrucción generada en el *fetch*.
- ***debug_read_address***: se trata de otro bus de 32 bits por el que la memoria puede recibir direcciones arbitrarias en cualquier instante de tiempo con fines de depuración.
- ***address@bundle***: la utilidad de este puerto, definido dentro del *bundle*, es indicar una dirección de memoria, cualquiera, para leerla o escribir en ella
- ***write_data@bundle***: es un bus de 32 bits mediante el cual se indica un dato a escribir en memoria
- ***write_enable@bundle***: una señal con la que habilitar o deshabilitar la escritura en memoria
- ***write_strobe@bundle***: tal y como puede verse en la Figura 4.2, donde se muestra el código con el que se implementa la clase *RAMBundle*, el tipo de dato del puerto de entrada *write_strobe* es un vector. Cada elemento de este vector es un booleano que indica si la sección n -ésima de la palabra a escribir en memoria, donde $0 \leq n < \text{Parameters.DataWidth}$, con $\text{Parameters.DataWidth} = 4$, deberá o no serlo. Por ejemplo, en las instrucciones de escritura de bytes individuales, el vector con el que se define el ‘strobe’ deberá tener la siguiente forma para indicar que deberá escribirse el último de los 4 bytes que conforman el dato: {0001}.

Señales de salida

- ***instruction***: la instrucción leída en la dirección *instruction_address*.
- ***debug_read_data***: el dato leído en la dirección *debug_read_address*.
- ***read_data@bundle***: el dato leído en la dirección *address@bundle* con la máscara indicada en el *strobe*.

```

1 class RAMBundle extends Bundle {
2   val address      = Input(UInt(Parameters.AddrWidth))
3   val write_data   = Input(UInt(Parameters.DataWidth))
4   val write_enable = Input(Bool())
5   val write_strobe = Input(Vec(Parameters.WordSize, Bool()))
6   val read_data    = Output(UInt(Parameters.DataWidth))
7 }

```

Figura 4.2: Implementación de la clase RAMBundle

Sobre las memorias que proporciona Chisel, estas son direccionables a palabra, no a byte. Esta decisión puede deberse a la necesidad de dotar a la librería con la capacidad de generar memorias de cualquier tipología y tipo de direccionamiento, en función del tipo de dato con que se inicialice la memoria en el constructor, y en función de la máscara que se emplee en los accesos a estas memorias. Por ejemplo, una memoria inicializada del siguiente modo: `SyncReadMem(4096, Vec(4, 32))`, daría como resultado una memoria con 4096 entradas donde, cada entrada, albergaría 4 palabras de 32 bits cada una. Puede intuirse que este mecanismo de generación de memorias y acceso a las mismas podría posibilitar la construcción de caches, indicando el tag en el parámetro de dirección de los métodos de lectura/escritura.

4.1.3. InstructionROM y ROMLoader

Puede verse en el diagrama de la Figura 4.1 que el computador lo componen, además del núcleo y la memoria, dos piezas adicionales: *InstructionROM* y *ROMLoader*. Estos son dos módulos involucrados en la carga de binarios en memoria. El primero define internamente una rutina con la que se traslada el contenido del fichero binario resultante de un proceso de compilación cruzada, a un fichero de texto codificado en ASCII con el que puede inicializarse una memoria de Chisel, definida también a nivel interno dentro del módulo. Posteriormente, la lógica que define el comportamiento del módulo *ROMLoader* se encarga de trasladar, palabra a palabra, los contenidos de la ROM de instrucciones a la memoria de la CPU, a partir de un punto de entrada recibido como parámetro.

Por último, el módulo *ROMLoader* cuenta con una señal de salida con la que indica al módulo superior *TopModule* que la carga del programa en memoria ha finalizado, con lo que la ejecución podría dar comienzo. Mientras la carga del programa sigue en curso, el *bundle* de la memoria (*Memory*) permanece conectado a un bundle definido dentro de *ROMLoader*, de modo que pueda hacerse el volcado del contenido de la memoria definida dentro de *InstructionROM*, a la memoria que empleará más adelante el core. Cuando el cargador finaliza su trabajo, se desacopla de la memoria y esta se conecta al core ya con el programa cargado y listo para ejecutar.

En la Figura 4.3 se proporciona el fragmento del código de la clase *TOPModule* con el que se construye la lógica que define el comportamiento descrito en este último párrafo. Puede verse que el acoplamiento entre los *bundles* del cargador y la memoria, o entre esta y el núcleo, se realiza por medio del operador '`<>`', con el que se realiza el conexionado de señales con el mismo nombre.

4.1.4. CPU

Este módulo constituye uno de los grandes bloques que conforma el computador, el núcleo, y en su interior se describe el conexionado entre los distintos bloques que modelan cada una de las etapas por las que pasa una instrucción cuando es leída de la memoria. El funcionamiento de estas etapas se describirá en los subapartados siguientes a este.

Además de este conexionado, se define una interfaz de entrada-salida con los siguientes puertos:

```

1 when(!rom_loader.io.load_finished) {
2   rom_loader.io.bundle <> mem.io.bundle
3   cpu.io.memory_bundle.read_data := 0.U
4 }.otherwise {
5   rom_loader.io.bundle.read_data := 0.U
6   cpu.io.memory_bundle <> mem.io.bundle
7 }
8

```

Figura 4.3: Implementación de la clase RAMBundle

Puertos de entrada

- **instruction**: la instrucción leída de la memoria, en la dirección que genera el módulo *Instruction-Fetch*.
- **instruction.valid**: una señal que controla el flujo de ejecución de nuevas instrucciones, en tanto que puede detener su lectura, paralizando el pipeline. Esta entrada se conecta al puerto de salida *load_finished* del cargador, con el que se indica que la carga de instrucciones en la memoria principal del computador ha finalizado, pudiendo dar comienzo a la ejecución del programa.
- **debug_read_address**: un bus de 5 bits por el cual puede indicarse el índice de un registro cuyo valor quiera conocerse. Esta dirección es recibida por medio de su correspondiente puerto de depuración en el módulo superior (*TopModule*).

Puertos de salida

- **debug_read_data**: el puerto de depuración donde el núcleo deposita el contenido del registro solicitado.
- **instruction.address**: la dirección de la siguiente instrucción generada en el *fetch*.
- **device.select**: el computador puede tener conectados hasta 8 dispositivos mapeados en memoria, cada uno de los cuales se identifica por medio de una etiqueta de 3 bits ($\log_2 8 = 3\text{bits}$), y es este el puerto por medio del cual se indica el dispositivo con cuyo espacio de memoria se quiere interactuar. En la práctica este puerto carece de utilidad, puesto que el único dispositivo con cuya memoria puede trabajarse desde el núcleo es la propia memoria principal. No obstante, queda claro que este mecanismo resultaría muy conveniente si se quisiera dotar al computador de comunicación de entrada-salida con una mayor variedad de dispositivos.

4.1.5. InstructionFetch

En este módulo se describe el comportamiento de la fase de *fetch*, donde se avanza el contador de programa a la siguiente instrucción sumándole 4 unidades¹, siempre y cuando se de alguna de las dos circunstancias siguientes:

- Que el cargador haya terminado de volcar el contenido de la ROM de instrucciones en la memoria principal de la CPU.
- Que el decodificador no haya determinado en el ciclo anterior que la instrucción en ese ciclo era un salto que debía tomarse.

En el primer caso el contador de programa permanecerá inmutable hasta el momento en que el cargador comunique la finalización del proceso de volcado y, en el segundo caso, el contador de programa tomará el valor del destino del salto.

¹Recordemos que se trata de un sistema 32 bits direccionable a byte, por lo que la siguiente instrucción se encontrará alejada 32 bits (4 bytes) respecto de la actual

Señales de entrada

- ***jump_flag_id***: es un bit que indica si el valor del contador de programa deberá ser igual al valor actual más cuatro unidades o, por el contrario, la dirección de destino de un salto que debe ser tomado.
- ***jump_address_id***: una señal de 32 bits por donde se recibe la dirección de destino calculada en la ejecución de las instrucciones de salto.
- ***instruction_read_data***: la instrucción leída de la memoria en la posición del contador de memoria.
- ***instruction_valid***: la señal con la que el cargador indica que su trabajo ha finalizado.

Señales de salida

- ***instruction_address***: la dirección de la instrucción a leer de la memoria en el siguiente ciclo.
- ***instruction***: la instrucción leída de la memoria en el ciclo actual. Se hace un *passthrough*, es decir, una asignación directa, entre este puerto y el puerto *instruction_read_data*.

Originalmente el código de este módulo se encontraba incompleto, puesto que carecía de la implementación de la lógica que establece el siguiente valor del contador de programa, es decir, el destino de un salto o la dirección de la siguiente instrucción en el programa. Esta lógica, mostrada en el código de la Figura 4.4, consiste en un multiplexor cuyo selector se conecta a la señal *jump_flag_id*, y cuyas dos entradas son, por un lado, el destino de un salto recibido por el puerto *jump_address_id* y, por otro lado, el valor del contador de programa más cuatro unidades (recordemos que se trata de una arquitectura de 32 bits).

```
1 pc := Mux(io.jump_flag_id, io.jump_address_id, pc + 4.U)
2
```

Figura 4.4: Conexión del contador de programa a la salida del multiplexor que determina su valor a cada ciclo en función de los saltos que se producen en el flujo de ejecución de los programas cargados en el computador

4.1.6. InstructionDecode

Como su nombre indica, esta es la clase que modela la lógica encargada de realizar la decodificación de las instrucciones recibidas desde el módulo de *fetch* para elaborar el conjunto de señales que componen la palabra de control, por medio de las cuales se gobernará el comportamiento del resto de unidades funcionales en el pipeline, de modo que estas actúen conforme al significado de las instrucciones decodificadas.

Se trata quizás del módulo con mayor complejidad de entre todos de los que se compone el pipeline, y esto se debe al elevado número de casuísticas que deben contemplarse en la generación de la palabra de control, que son el resultado directo de la variabilidad en cuanto a formatos de instrucciones dentro del ISA, así como de la variabilidad en el conjunto de valores posibles que pueden tomar los campos definidos en estos formatos.

Señales de entrada

- ***instruction***: la instrucción leída en la fase de *fetch*.

Señales de salida

- ***regs_reg{1,2}_read_address***: las direcciones de los registros cuyos valores se emplearán en el cómputo de los resultados de instrucciones aritmético-lógicas, o en la determinación del cumplimiento de la condición de salto en las instrucciones de salto condicional.
- ***ex_immediate***: inmediato son signo, extendido a 32 bits.
- ***ex_aluop{1,2}_source***: señales de control mediante las cuales se selecciona la procedencia de los operandos de la ALU.
- ***memory_{read,write}_enable***: señales de control mediante las cuales se habilita la lectura o escritura en la memoria.
- ***wb_reg_write_source***: señal de control con la que se selecciona el origen del dato a escribir en el banco de registros en la fase de *writeback*.
- ***reg_write_enable***: señal de control mediante la cual se habilita la escritura en el banco de registros.
- ***reg_write_address***: la dirección del registro en el que debe escribirse un resultado en la fase de *writeback*.

Dentro de este módulo tan solo fue preciso implementar la lógica correspondiente a la habilitación de la lectura o escritura en memoria, así como de la selección del origen del resultado a escribir en la fase de *writeback*, tal y como puede verse en el código expuesto en la Figura 4.5. Esta lógica se materializa por medio de un par de multiplexores: el primero de ellos gobierna la habilitación de la lectura en memoria y, el segundo, la escritura en memoria.

```
1  io.memory_read_enable := Mux(  
2      opcode === InstructionTypes.L,  
3      true.B,  
4      false.B  
5  )  
6  io.memory_write_enable := Mux(  
7      opcode === InstructionTypes.S,  
8      true.B,  
9      false.B  
10 )
```

Figura 4.5: Generación de las señales de habilitación de lectura/escritura en memoria

4.1.7. InstructionExecute

La clase *Execute* modela el comportamiento del hardware implicado en las operaciones que se llevan a cabo en la fase de ejecución y que son, por un lado, el cómputo de resultados de operaciones aritmético-lógicas en la ALU² y, por otro lado, la evaluación de las condiciones de salto junto con el cómputo del siguiente contador de programa en caso de resultar positivas esas evaluaciones.

La ALU se construye dentro del módulo *Execute* por medio de una instancia de la clase *ALU*, la cual cuenta con tres puertos de entrada (operación y primer y segundo operandos), y uno de salida (resultado). Además, su comportamiento es gobernado en base a las salidas generadas por una instancia de la clase *ALUControl*, por medio de cuya lógica se infiere el tipo de operación que deberá llevarse a cabo, en función de los valores que tomen los campos *opcode*, *funct3* y *funct7* en la instrucción.

A continuación se detallan las señales que conforman la entrada-salida del módulo, así como la función que desempeñan dentro de este.

²Arithmetic-Logic Unit: Unidad Aritmético-Lógica

Señales de entrada

- ***instruction***: la instrucción completa, de la cual se extraerán los campos *opcode*, *funct3* y *funct7*.
- ***instruction_address***: la dirección de la instrucción en ejecución. Este valor se empleará en el cómputo de direcciones efectivas de salto, así como del valor resultante de la ejecución de la instrucción *auipc*.
- ***reg1_data***: el valor leído en la dirección del primer registro fuente.
- ***reg2_data***: el valor leído en la dirección del segundo registro fuente.
- ***immediate***: el inmediato codificado en la instrucción, previamente extendido a 32 bits en la fase de decodificación.
- ***aluop1_source***: una señal de un bit con la que se indica cuál deberá ser el valor empleado como primer operando en la ALU. Esta señal controla la salida de un multiplexor cuyas entradas son la dirección de la instrucción y el valor leído en la dirección del primer registro fuente.
- ***aluop2_source***: una señal de un bit con la que se indica cuál deberá ser el valor empleado como segundo operando en la ALU. Esta señal controla la salida de un multiplexor cuyas entradas son el inmediato y el valor leído en la dirección del segundo registro fuente.

Señales de salida

- ***mem_alu_result***: el valor calculado en la ALU.
- ***if_jump_flag***: una señal de un bit con la que se indica el resultado de la evaluación de la condición de salto.
- ***if_jump_address***: en caso de resultar positiva la evaluación de la condición de salto, la dirección efectiva de este.

4.1.8. Memory Access

El módulo *MemoryAccess* incorpora la lógica por medio de la cual se gestionan los accesos a la memoria de datos en todas las variantes admitidas como parte de la especificación base, es decir, lecturas/escrituras de palabras completas, mitades o bytes individuales, con o sin extensión de signo.

A continuación se indicarán los puertos que componen la entrada-salida del módulo, así como su propósito y el uso que se hace de los datos que albergan. No obstante cabe mencionar que, como parte de la entrada-salida del módulo, se crea una instancia de la clase *RAMBundle* dentro de la cual se definen los puertos con los que poder interactuar con la memoria, tal y como se indicó en el apartado 4.1.2.

Señales de entrada

- ***alu_result***: la dirección en la cual se va a efectuar el acceso. Este mismo valor se pasa como entrada al bundle en su puerto *address*.
- ***reg2_data***: cuando va a realizarse una escritura, el dato a escribir.
- ***memory_read_enable***: un bit que habilita/deshabilita la lectura de datos en memoria.
- ***memory_write_enable***: un bit que habilita/deshabilita la escritura de datos en la memoria.
- ***funct3***: indica la longitud del dato con la que operar (palabra completa, mitad o byte).

Señales de salida

- ***wb_memory_read_data***: cuando se realiza una operación de lectura, el dato leído.

La memoria es direccionable a byte y, a partir de los dos³ últimos bits de la dirección puede computarse la posición del byte al cual se desea acceder. Ese valor, almacenado en la señal interna *mem_address_index*, se emplea del modo siguiente:

- Cuando se trata de **lecturas** de bytes individuales, con o sin extensión de signo, el índice se emplea para extraer la sección de 8 bits correspondiente de la palabra leída por medio del bundle. En el caso de las lecturas de media palabra, cuando el valor del índice es igual a 0 se lee la mitad menos significativa y, en cualquier otro caso, la más significativa.
- En las operaciones de **escritura** el índice se emplea en la construcción del *strobe* o máscara indicado en el bundle por medio del puerto *write_strobe*. En las escritura de bytes individuales se configura la máscara con un 1 en la posición del índice y el resto a 0 y, en las operaciones de escritura de media palabra, cuando el valor del índice es igual a 0 los dos bits menos significativos de la máscara se configuran con el valor 1; en cualquier otro caso se configurarían con valor 1 los dos bits más significativos.

4.1.9. WriteBack

El módulo *WriteBack* consta de un multiplexor por medio del cual se selecciona el origen del dato a escribir en el banco de registros, bien sea un valor computado en la ALU, un dato leído de la memoria o el siguiente valor del contador de programa.

Señales de entrada

- ***alu_result***: el valor computado en la ALU.
- ***memory_read_data***: el dato leído de la memoria.
- ***instruction_address***: la dirección de la instrucción en curso.
- ***regs_write_source***: la señal de control que determina la salida en la fase de writeback.

Señales de salida

- ***regs_write_data***: el valor a escribir en el banco de registros.

${}^3\log_2 \left(\frac{32 \text{ bits (longitud de palabra)}}{8 \text{ bits (1 byte)}} \right) = 2$

5 El procesador segmentado

La segmentación (*pipelining*) es una técnica por medio de la cual la arquitectura de un computador se divide en múltiples etapas, cada una de las cuales, se corresponde con una de las fases experimentadas por una instrucción desde que es leída de memoria hasta que sus resultados son escritos en el banco de registros. Este enfoque permite que distintas instrucciones ocupen distintas etapas de manera concurrente, explotando así el paralelismo a nivel de instrucción, y requiere introducir en la arquitectura registros de etapa con los que almacenar de manera temporal el contexto de una instrucción al final de cada etapa, es decir, el conjunto de datos y señales de control asociados a la instrucción, los cuales serán transferidos a la etapa inmediatamente posterior en el siguiente ciclo de reloj.

Puede verse en la Figura 5.1 un diagrama en el que se compara el flujo de ejecución de un computador monociclo frente al de uno segmentado.

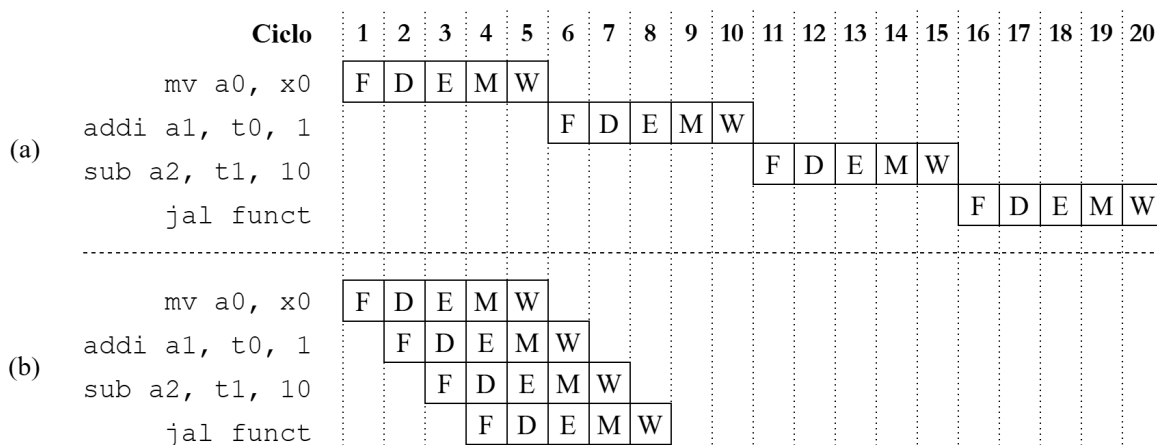


Figura 5.1: Ejecución de un programa en un computador con diseño monociclo (a) y segmentado (b)

Esta técnica conlleva una mayor complejidad arquitectónica y el surgimiento de una serie de riesgos o *hazards* generados bien por las dependencias de datos entre las instrucciones de un programa (ver Figuras 5.2 y 5.3), o bien por la ejecución de instrucciones de control que pueden o no alterar el flujo de ejecución de este (ver Figura 5.4). En función de las decisiones de diseño que se tomen para abordar la problemática que representan estos fenómenos, su aparición desembocará en la introducción de detenciones (*stalls*) en el flujo de ejecución, o la necesidad de incluir en el diseño mecanismos adicionales como el *forwarding*, consistente en el conexionado de determinadas señales de datos o de control hacia etapas anteriores en el pipeline.

En la Figura 5.2 aparecen identificados los distintos tipos de dependencias de datos que pueden darse durante la ejecución de un programa. La dependencia RAW (a) se da cuando uno de los operandos de una instrucción es un registro (`x1`, señalado en color morado) cuyo valor deberá ser previamente modificado

add x1 , x2, x3 sub x4, x1 , x5 (a)	add x1 , x2, x3 add x1 , x4, x5 (b)	add x4, x1 , x2 add x1 , x3, x5 (c)
---	---	---

Figura 5.2: Ejemplos de riesgos de datos. (a) RAW (*Read After Write*): lectura tras escritura. (b) WAW (*Write After Write*): escritura tras escritura. (c) WAR (*Write After Read*): escritura tras lectura

por otra instrucción separada de la primera 3 ciclos o menos. Esto es así ya que la primera instrucción, la que lee el dato, deberá poder observar ese dato actualizado de modo que pueda elaborar un resultado coherente con el orden del programa. Es decir que, en un principio, la segunda instrucción no debería poder leer sus operandos (en su *decode*) hasta la escritura (*writeback*) del resultado calculado por la primera. Se ejemplifica esta situación por medio del diagrama presentado en la Figura 5.3.

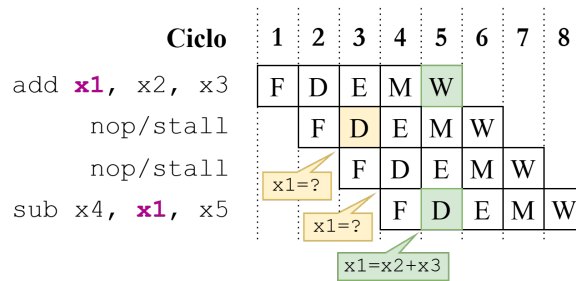


Figura 5.3: Flujo de ejecución de dos instrucciones implicadas en una dependencia RAW. Se señala en color verde el instante a partir del cual la segunda instrucción podrá leer el dato actualizado en el registro **x1** (ciclo 5).

Los otros dos tipos de dependencias presentados, señalados con las letras *b* y *c*, no representan un riesgo real ya que dos instrucciones pueden leer o escribir de manera consecutiva sobre el mismo registro sin que ello suponga la pérdida de coherencia en el dato, al igual que sucede con las escrituras sobre un registro que fue previamente leído.

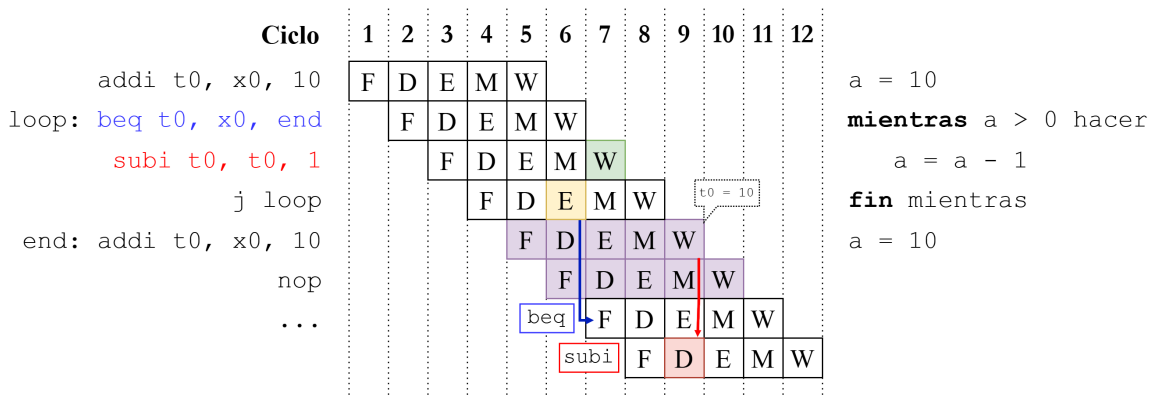


Figura 5.4: Representación del flujo de ejecución de un programa sobre una arquitectura segmentada sin mecanismos para resolver riesgos de control.

Sobre el ejemplo de riesgo de control presentado en la Figura 5.4, el resultado de la instrucción de resta (`subi`) se escribe (writeback señalado en color verde) en el banco de registros ántes incluso de evaluar la condición en la instrucción de salto, dejando el valor del registro `t0` en un estado inconsistente. Además, dado que el destino del salto de la instrucción `jál` no se calculará hasta su ejecución, habrá dos ciclos en los que se introducirán las dos instrucciones inmediatamente posteriores (señaladas en color morado), una de las cuales reinicia el valor de la variable `a` haciendo que el bucle no tenga final. El resultado de la instrucción `subi` en la primera iteración se sobrescribe con el del segundo `addi`.

Por último, y a pesar de las limitaciones comentadas, se debe destacar que la segmentación de instrucciones permite obtener un speedup teórico próximo al número de etapas en que se haya decidido segmentar la arquitectura, teniendo siempre en cuenta que aspectos como la presencia y resolución de los riesgos mencionados, la latencia en los accesos a memoria o el tiempo desigual entre los caminos críticos de las diferentes etapas, impedirán alcanzar ese rendimiento máximo ideal.

5.1. Arquitectura de un registro de etapa

La función de un registro de etapa es almacenar de manera transitoria el conjunto de señales de control y datos emitidos al término de una etapa de modo que, en el siguiente ciclo de reloj, esas mismas señales puedan ser transferidas a la etapa inmediatamente posterior en el pipeline. De este modo se consigue desacoplar temporalmente la ejecución de las distintas etapas permitiendo que varias instrucciones ocupen el pipeline de manera concurrente.

Desde el punto de vista de la implementación en Chisel, un registro de etapa como el mostrado en la Figura 5.5 se compone, en primer lugar, de un bundle en el que se definen las entradas y salidas de este. Estas se corresponderán con el conjunto de señales de control y datos que deban avanzar de una etapa a la siguiente. Posteriormente, cada entrada de datos/control se conectará a la entrada de un registro específico de esa señal y, la salida del registro, a su correspondiente salida en el bundle.

```

1  class FD extends Module {
2
3      val io = IO(new Bundle {
4          val f_instruction      = Input(UInt(Parameters.InstructionWidth))
5          val f_instruction_address = Input(UInt(Parameters.AddrWidth))
6
7          val d_instruction      = Output(UInt(Parameters.InstructionWidth))
8          val d_instruction_address = Output(UInt(Parameters.AddrWidth))
9      })
10
11     val instruction      = RegInit(0.U(Parameters.InstructionWidth))
12     val instruction_address = RegInit(0.U(Parameters.AddrWidth))
13
14     instruction      := io.f_instruction
15     instruction_address := io.f_instruction_address
16
17     io.d_instruction      := instruction
18     io.d_instruction_address := instruction_address
19 }

```

Figura 5.5: Implementación inicial del registro de etapa que separa las fases de *fetch* y decodificación.

5.2. Segmentando el núcleo en dos etapas

El primer paso hacia la segmentación del núcleo en las cinco etapas marcadas como objetivo del trabajo consistió en segmentarlo primeramente en dos, aislando la fase de *fetch* respecto de las unidades funcionales implicadas en el resto de etapas.

Esta división no conlleva la aparición de riesgos de datos puesto que tanto la escritura como la lectura de estos forman parte de la misma fase dentro del ciclo de procesamiento de las instrucciones. Sin embargo, esta segmentación sí da pie a que, durante la ejecución de un programa, puedan aparecer riesgos de control tal y como se muestra en el ejemplo presentado en la Figura 5.6, donde la ausencia de mecanismos que logren mitigar el riesgo de control provoca la lectura de la instrucción señalada en color rojo antes de que se haya evaluado siquiera la condición en la instrucción de salto, ejecutando así la instrucción de resta y depositando en el registro `x1` un valor incoherente con la lógica del programa.

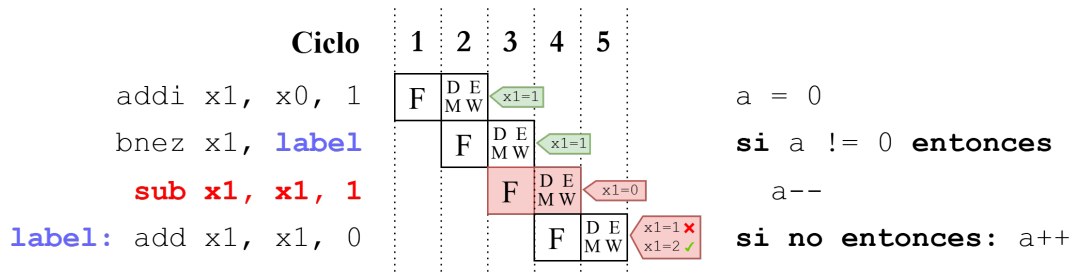


Figura 5.6: Representación del flujo de ejecución de un programa sobre una arquitectura segmentada en dos etapas sin mecanismos para resolver riesgos de control.

Para verificar la existencia o no del riesgo de control tras la introducción en la arquitectura del primer registro de segmentación, se desarrolló un sencillo programa en lenguaje ensamblador el cual incorpora instrucciones de salto condicional, salto incondicional y una llamada a función. Este programa (cuyo código se muestra en la Figura 5.7) fue compilado, cargado en memoria y ejecutado en el computador simulado por medio del test de Chisel mostrado en la Figura 5.8. El test consta, en primer lugar, de un bucle de inicialización en el que se espera a que el cargador lea el contenido del binario y lo deposite en la memoria del computador. Posteriormente, se ejecuta un segundo bucle en el que se hace avanzar el reloj 100 ciclos, a priori, suficientes como para recoger la ejecución completa del programa en la traza VCD resultante de la ejecución del test.

El primero de los problemas que se observó al examinar la traza generada por el test fue la falta de sincronismo entre el valor del contador de programa observado en una etapa determinada, y la instrucción observada en esa misma etapa, tal y como aparece señalado en color rojo en la Figura 5.9. En esa sección de la traza se muestran los cambios en el valor de las siguientes señales:

- `instr_addr@fetch`: la dirección de la instrucción presente en la fase de fetch.
- `instr_addr@DEMW`: la dirección de la instrucción en la fase siguiente al fetch.
- `instr@fetch`: la instrucción leída en la última fase de fetch.
- `instr@DEMW`: la instrucción leída en el anterior fetch.
- `fetch_io.jump_flag`: la señal que indica si la evaluación de una condición es positiva y debe leerse por tanto en el siguiente ciclo, no la instrucción siguiente al contador de programa actual, si no el *target* del salto. Tanto el *target* como esta señal se originan en la fase siguiente al fetch, como parte de la fase de ejecución (módulo *Execute*).
- `ex_io.immediate`: el offset empleado en el cálculo de la dirección efectiva de salto, también como parte de la fase de ejecución.
- `ex_io.instr_addr`: la dirección base del salto.
- `fetch_io.jump_addr`: la dirección efectiva del salto. Esta señal llega al módulo *InstructionFetch* desde *Execute*.

Código máquina	Ensamblador	Pseudocódigo
<pre> 00000000 <_start>: 0: 00000313 4: 00400393 00000008 <loop_start>: 8: 0063ca63 c: 00030513 10: 010000ef 14: 00050313 18: fffff06f 0000001c <loop_end>: 1c: 0000006f 00000020 <suma_uno>: 20: 00050293 24: 00128293 28: 00028513 2c: 00008067 </pre>	<pre> li t1,0 li t2,4 blt t2,t1,loop_end mv a0,t1 jal suma_uno mv t1,a0 j loop_start j loop_end mv t0,a0 addi t0,t0,1 mv a0,t0 ret </pre>	<pre> t1 = 0 t2 = 4 mientras t1 < t2 t1 = suma_uno(t1) función suma_uno(x) retornar x + 1 </pre>

Figura 5.7: Desensamblado, código ensamblador y pseudocódigo del programa empleado en la depuración de la arquitectura.

```

1 class SegRegFDTest extends AnyFlatSpec with ChiselScalatestTester {
2   behavior.of("Pipelined CPU")
3   it should "run a simple program" in {
4     test(new TestTopModule("simple_subroutine.asmbin"))
5       .withAnnotations(TestAnnotations.annos) { c =>
6         while(!c.io.instruction_valid.peek().litToBoolean) {
7           c.clock.step()
8           c.io.mem_debug_read_address.poke(4.U)
9         }
10
11         for (i <- 1 to 100) {
12           c.clock.step()
13           c.io.mem_debug_read_address.poke((i * 4).U)
14         }
15
16         c.io.regs_debug_read_address.poke(6.U) // x6 => t1
17         c.io.regs_debug_read_data.expect(5.U)
18       }
19   }
20 }

```

Figura 5.8: Código del test con el que se pone a prueba el funcionamiento de un programa sencillo sobre el núcleo segmentado en dos fases.

La consecuencia que esta falta de sincronismo tiene sobre el flujo de ejecución de los programas es que las instrucciones de salto no pueden realizar correctamente el cómputo de la dirección efectiva de este, al contar en la fase de ejecución con el contador de programa de la instrucción siguiente al salto, y no la del propio salto. Tal y como se observa en la traza de la Figura 5.9, al comienzo del ciclo marcado en la posición en que se encuentra el indicador vertical (43 ps), la instrucción `jal suma_uno (0x010000EF @`

0x1010¹, ver señal `instr@DEMW`), observa los siguientes operandos en el cálculo del destino del salto:

- `ex_io_immediate`: 0x10
- `instr_addr@DEMW`: 0x1014

Puede observarse que la dirección base del salto no es la correspondiente a la instrucción `jal`, si no la de la instrucción `mv` que la sucede en el programa, provocando que el salto no sea al comienzo de la subrutina `suma_uno`, si no a la segunda instrucción dentro de esta.

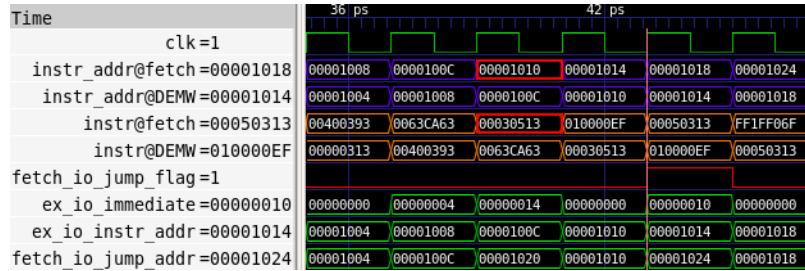


Figura 5.9: Sección de la traza obtenida tras la ejecución del test mostrado en la Figura 5.7

El diseño original emplea como memoria una instancia de la clase *SyncReadMem*. Este tipo de memoria no permite realizar lecturas de manera asíncrona, lo cual implica que, cuando se solicita una lectura, el dato leído no sale de la memoria hasta el siguiente flanco ascendente de reloj, impidiendo que se mantenga la sincronía entre las instrucciones en cada fase y la dirección de la instrucción observada en esa misma fase. En el computador objeto de este trabajo, este comportamiento de la memoria se materializa en que, cuando se envía la dirección de una instrucción para leerla, esa lectura no será efectiva hasta el ciclo siguiente al envío de la dirección. De ahí que, por ejemplo, la dirección de una instrucción en la fase de fetch (señal `instr_addr@fetch`) no se corresponda con la instrucción observada en esa misma fase (señal `instr@fetch`), si no con la instrucción inmediatamente anterior a la solicitada.

Para solucionar esta problemática se cambia, dentro de la clase *Memory*, el tipo de memoria a una de la clase *Mem*, de forma que la lectura del dato se lleve a cabo en el instante mismo en que se solicita. De este modo, tal y como puede observarse en la traza presentada en la Figura 5.10, el valor de la dirección de la instrucción en cada fase se corresponde con la de la instrucción que se está procesando en esa misma fase, de tal modo que ahora el cálculo del *target* en la instrucción de salto sea correcto ($0x1010$ (`ex_io_instr_addr`) + $0x10$ (`ex_io_immediate`) = $0x1020$ (`fetch_io_jump_addr`) → `suma_uno`).

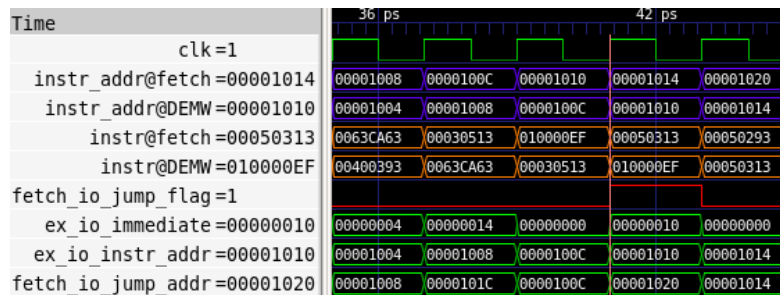


Figura 5.10: Misma sección de la traza pero con una instancia de la clase *Mem* como entidad de memoria en lugar de *SyncReadMem*.

¹El punto de entrada de los programas se encuentra en la dirección 0x1000. La dirección de la instrucción es el valor del punto de entrada sumado al offset mostrado en la primera columna de la Figura 5.7

A pesar de esta corrección, puede observarse que el flujo de ejecución del programa sigue siendo erróneo ya que, siguiendo a la instrucción de salto (tras la marca vertical), en la segunda fase logra introducirse y ejecutarse la instrucción inmediatamente posterior, la cual es un `mv t1, a0` (`0x00050313 @ 0x1014`). En el caso concreto de la primera iteración del bucle, la ejecución de esta instrucción es inócua ya que en ese punto los registros `t1` y `a0` albergan el mismo valor. No obstante a esto, la inserción de la instrucción siguiente al salto toma una relevancia significativa cuando se trata de otra instrucción de salto, tal y como ocurre tras el salto incondicional al final del bucle. La ejecución de este segundo salto, el cual forma parte del bucle de final de programa, provoca que este llegue a término tras la primera iteración (véase Figura 5.9).

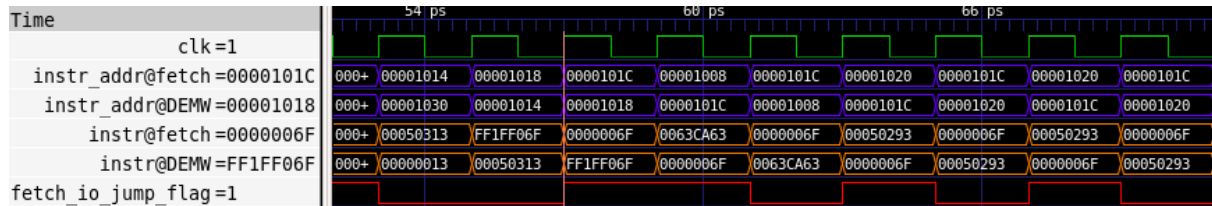


Figura 5.11: Sección de la traza en la que se muestra la pronta ejecución del bucle final.

La solución final para resolver esta problemática consiste en la introducción de un par de modificaciones en la fase de fetch para hacer que la arquitectura introduzca una operación `nop` tras cada instrucción de salto en cuya segunda fase se determine que este deba ser tomado. Estas modificaciones se realizan sobre la lógica que hace avanzar el contador de programa.

En la nueva implementación, tal y como se observa en el `diff` presentado en la Figura 5.12, se introduce un `nop` en el pipeline siempre y cuando se cumplan las dos condiciones siguientes: en primer lugar, que el cargador no haya terminado su trabajo y, en segundo lugar, que en la segunda fase de una instrucción de salto se haya determinado que este deba ser tomado (originalmente solo se contemplaba la primera condición).

```
val pc = RegInit(ProgramCounter.EntryAddress)

- when(io.instruction_valid) {
-   pc          := Mux(io.jump_flag_id, io.jump_address_id, pc + 4.U)
-   io.instruction := io.instruction_read_data
+ when(io.instruction_valid && !io.jump_flag_id) {
+   pc          := pc + 4.U
+   io.instruction := io.instruction_read_data
+ }.otherwise {
-   pc          := pc
+   pc          := io.jump_address_id
+   io.instruction := 0x00000013.U
+ }

io.instruction_address := pc
```

Figura 5.12: Comparación entre la implementación original del módulo `InstructionFetch` y su adaptación para resolver riesgos de control.

5.3. Segmentando el resto del núcleo

En un primer momento se optó por aplicar un enfoque incremental en lo que respecta a la inserción de los registros de etapa. No obstante, este enfoque se aplicó tan solo en la integración del primer registro de etapa para separar la fase de *fetch* del resto del pipeline. Esto fue así puesto que, de haber seguido integrando registros de etapa uno a uno, verificando el funcionamiento del núcleo con dos, tres y cuatro registros, el proceso de desarrollo se hubiera dilatado en el tiempo más allá de lo deseado inicialmente ya que, para cada una de las integraciones de un registro de segmentación a partir de aquel que separa las fases de *fetch* y decodificación, hubiera sido preciso diseñar una solución distinta al problema que generan las dependencias de datos y de control entre las instrucciones que atraviesan el pipeline.

Es por ello por lo que se consideró que resultaría más eficiente integrar los registros de segmentación siguientes al de *fetch*-decodificación a la vez, desarrollando una única solución al problema generado por las dependencias de datos y de control.

Esta solución se ha materializado en el código de la clase *HazardUnit*, la cual modela la lógica que gobierna el comportamiento de una unidad de amenazas. El cometido de esta pieza es observar el estado de la arquitectura a cada ciclo para identificar y resolver las dependencias de datos o de control que pudieran originarse durante la ejecución de los programas. A continuación se detalla el método empleado en la resolución de los riesgos de datos del tipo RAW y de control, así como las señales de la unidad de amenazas involucradas en el proceso de detección y mitigación del riesgo.

5.3.1. Resolución de riesgos de datos (RAW)

En el caso de una arquitectura segmentada en cinco fases, un riesgo de datos del tipo RAW se da cuando en un programa existe una pareja de instrucciones consecutivas para las cuales se cumple que, la segunda de ellas, posee como registro origen en alguno de sus operandos el registro de destino de la primera, pudiendo encontrarse separadas una distancia de 1 o 2 ciclos. Esta casuística queda ilustrada por medio del ejemplo presentado en la Figura 5.13, donde la instrucción `subi` debe esperar al *writeback* del `addi` para poder leer del registro `x10` un valor actualizado y consistente con la lógica del programa.

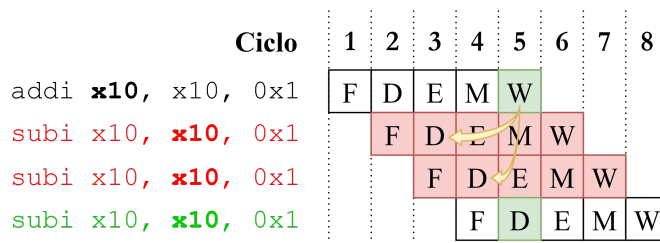


Figura 5.13: Ejemplo de una dependencia RAW en una arquitectura segmentada en cinco fases.

La solución (que no sea introducir *nops*) al problema que supone la ocurrencia de esta clase de escenarios pasa por la aplicación de una técnica denominada *forwarding*. Esta técnica consiste en adelantar el resultado de una instrucción, ya sea aritmético-lógica o una lectura de memoria, directamente a la fase de ejecución de las instrucciones que deban emplear ese dato como operando. Esto queda ejemplificado, también, en el diagrama de la Figura 5.13 por medio de las flechas de color amarillo: la primera instrucción `subi` podría ejecutar sin problemas y sin perder ni un solo ciclo si, desde el *writeback* del `addi` se le adelantara a su fase de ejecución el resultado a escribir en el registro `x10`, y de igual modo ocurriría con el segundo `subi`.

A continuación, se indicarán las soluciones propuestas al problema de las dependencias de datos del tipo RAW, tanto en el caso de las instrucciones aritmético-lógicas como en el caso de la instrucción de lectura de memoria:

Riesgos causados por la sucesión de instrucciones aritmético-lógicas

Las señales de la unidad de amenazas implicadas en la detección de esta clase de riesgos son:

- **rs1_ex**: la dirección del registro empleado como primer operando de la instrucción, observado en la fase de ejecución.
- **rs2_ex**: la dirección del registro empleado como segundo operando de la instrucción, observado en la fase de ejecución.
- **rd_mem**: la dirección del registro destino, observado en la fase de acceso a memoria.
- **rd_wb**: la dirección del registro destino, observado en la fase de escritura de resultados.
- **rd_wb**: la dirección del registro destino, observado en la fase de escritura de resultados.
- **rd_wb**: la dirección del registro destino, observado en la fase de escritura de resultados.

La unidad de amenazas gobierna, además, las señales de selección de dos multiplexores emplazados, cada uno, en una de las entradas de la ALU:

- **alu_rs1_src**: controla la selección del primer operando de la ALU, que puede ser un dato leído a partir del banco de registros por parte de la instrucción en fase de ejecución, o un dato adelantado desde la fase de memoria o de *writeback*.
- **alu_rs2_src**: exactamente igual que la anterior, pero aplicada al segundo operando de la ALU.

La lógica que rige el proceso de detección de las dependencias por parte de la unidad de amenazas es tal y como se indica en la Figura 5.14, siendo el código aplicable tanto en el caso en que la dependencia se diese con el primer registro operando, como si esta se diese con el segundo.

Más allá de lo auto-explicativo del pseudo-código, cabe reseñar el por qué de la condición en la que se evalúa si el registro origen es x0 y es que, de darse una dependencia RAW en la que este registro se encontrase involucrado, no tendría sentido realizar un *forwarding* de un registro cuyo valor es constantemente 0, por lo que se evita solventar esta falsa dependencia.

Código	Pseudo-código
<pre> io.alu_rs1_src := Mux(((io.rs1_ex == io.rd_mem) & io.reg_wr_enable_mem) & (io.rs1_ex != 0.U), RsFwSel.ALUResultMemFw, Mux(((io.rs1_ex == io.rd_wb) & io.reg_wr_enable_wb) & (io.rs1_ex != 0.U), RsFwSel.ALUResultWbFw, RsFwSel.RegFileData)) </pre>	<pre> # Detección dependencia RAW entre Ex y Mem si primer reg. origen @ Ex = reg. destino @ Mem y escritura regfile @ Mem habilitada y primer reg. origen @ Ex != x0 entonces 1er operando ALU = resultado adelantado desde Mem # Detección dependencia RAW entre Ex y Wb si no si primer reg. origen @ Ex = reg. destino @ Wb y escritura regfile @ Wb habilitada y primer reg. origen @ Ex != x0 entonces 1er operando ALU = resultado adelantado desde Wb si no entonces 1er operando ALU = dato leído por la instrucción </pre>

Figura 5.14: Código y pseudo-código de la lógica empleada en la detección y mitigación de riesgos de datos del tipo RAW en los que se encuentra involucrado el primer registro origen de una instrucción.

Dependencias en las que se encuentra involucrada la instrucción 'lw'

La presencia de la instrucción `lw` en una dependencia de datos del tipo RAW genera un caso especial que no puede resolverse por medio de la metodología seguida en el apartado anterior. Esto es debido a que el resultado de la instrucción `lw` no se genera hasta haber completado la lectura de un dato de memoria, lo cual se produce al término de la fase de memoria de la instrucción. En ese momento la instrucción dependiente inmediatamente posterior al `lw` ya habrá completado su fase de ejecución, pero lo habrá hecho con un operando erróneo.

La técnica con la que se resuelve la dependencia consiste en detener el procesamiento de las instrucciones en fases de *fetch* y decodificación cada vez que esta sea detectada. Esto se consigue reteniendo los datos de los registros de segmentación que separan las fases de *fetch*-decodificación, y decodificación-ejecución. Esta retención dura tan solo un ciclo, de tal modo que, cuando la instrucción dependiente que no es el `lw` pase a la fase de ejecución, la unidad de amenazas tratará la dependencia como si de cualquier otra dependencia de datos se tratase, adelantando desde el writeback del `lw` el valor leído de la memoria, a la entrada de la ALU que corresponda. Se proporciona en la Figura 5.15 una representación de la resolución del riesgo de datos por medio de esta técnica.

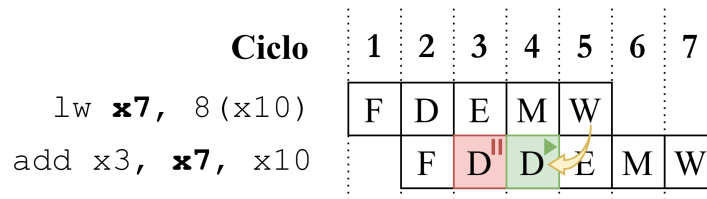


Figura 5.15: Flujo de ejecución de dos instrucciones implicadas en una dependencia de datos de tipo RAW, una de las cuales es la instrucción `lw`.

La solución implementada (Figura 5.16) ha consistido en hacer que la unidad de amenazas identifique, en primer lugar, si una instrucción en fase de ejecución va a provocar la escritura en el banco de registros de un dato leído de memoria. Esto se consigue observando el bit menos significativo de la señal `ex_regs_write_src`, la cual controla la salida del multiplexor en la fase de *writeback* con el que se recoge el dato a escribir en el banco de registros. La salida de este multiplexor podrá ser un resultado calculado en la ALU, la dirección de retorno de un salto o el dato leído en una posición de memoria, siendo en este último caso, y por implementación, el valor del bit menos significativo de la señal igual a 1. Esta es condición suficiente y necesaria para asegurar que la instrucción en fase de ejecución es un `lw`. A continuación, en la segunda condición de la sentencia 'y', se realiza la detección de la dependencia de datos RAW con la instrucción inmediatamente posterior, de igual modo que se hacía en los casos comentados en el apartado anterior.

El resultado de estas comprobaciones es la señal `lw_stall`, que gobierna la parada del contador de programa y los registros de segmentación indicados anteriormente.

```

1 val lw_stall = io.ex_regs_write_src(0) &
2               ((io.rs1_d === io.rd_ex) | (io.rs2_d === io.rd_ex))
3
4 io.pc_stall    := lw_stall
5 io.srFD_stall := lw_stall
6 io.srDE_flush := lw_stall

```

Figura 5.16: Cómputo de la señal `lw_stall`.

5.3.2. Resolución de riesgos de control

Una solución sencilla y eficaz al problema generado por los riesgos de control es limpiar el contenido de los registros de segmentación entre las etapas de fetch-decodificación y decodificación-ejecución cada vez que, en la fase de ejecución de una instrucción de salto, se determina que este deba ser tomado. De este modo, tal y como se observa en el ejemplo de la Figura 5.17, todas las instrucciones (máximo 2) que logren introducirse en el pipeline tras la instrucción de salto, no se procesarán completamente y su paso por la arquitectura habrá sido totalmente inocuo.

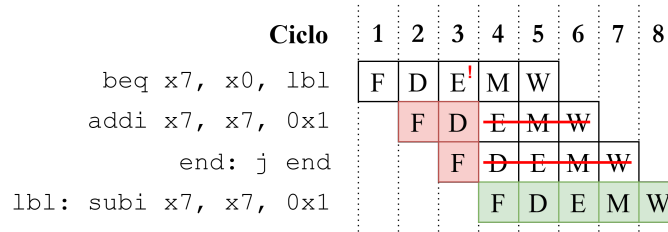


Figura 5.17: Diagrama ilustrativo del proceso de mitigación de los riesgos de control.

Esta solución se implementa por medio del fragmenteo de código de la unidad de amenazas mostrado en la Figura 5.18, donde se establece que el valor de la señal de borrado de los registros de segmentación indicados (señales `srDE_flush` y `srFD_flush`), será igual al valor del flag de salto observado en la fase de ejecución.

```

1 io.srDE_flush := lw_stall | io.ex_jump_flag
2 io.srFD_flush := io.ex_jump_flag

```

Figura 5.18: Configuración de la señal de borrado de los registros de segmentación que separan las fases de fetch-decodificación y decodificación-ejecución, conforme al valor del flag de salto en fase de ejecución.

6 Despliegue del diseño segmentado sobre FPGA

La última fase del proyecto consiste en la adaptación del diseño realizado para poder ser cargado en el chip FPGA de la placa de desarrollo Arty A7 100T. Además, se pretende realizar una prueba de concepto en la que se envíe un código de ejemplo en formato binario por medio de la interfaz serie disponible en la placa. Esto último requerirá incluir una serie de modificaciones en el diseño de la CPU de tal modo que la tarea de realizar la inicialización de la memoria ya no recaiga en el cargador, si no en un nuevo módulo que dote al computador de capacidad para comunicar, usando el protocolo UART, con un host externo del cual recibirá el código de prueba. Esta comunicación se iniciará en el host remoto por medio de la utilidad *TeraTerm*.

Se muestra en la Figura 6.1 el diagrama de la arquitectura global del computador, actualizado para sustituir la rom de instrucciones y el cargador (*InstructionROM* y *ROMLoader* respectivamente), por un nuevo módulo *UART*.

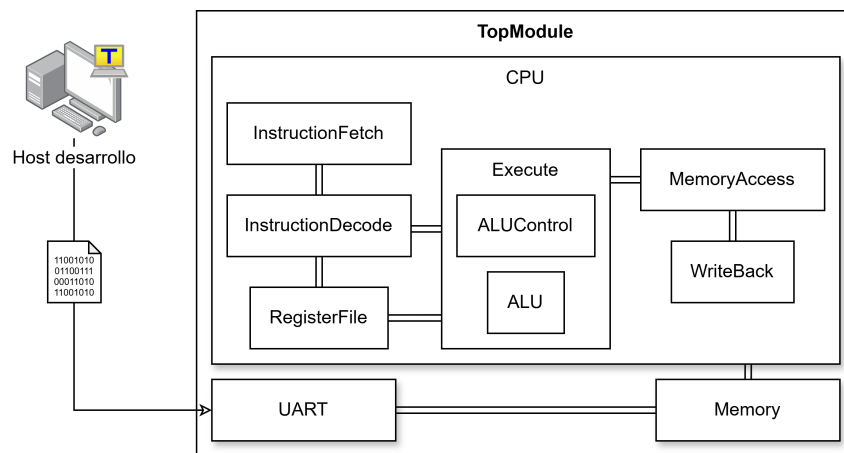


Figura 6.1: Diagrama de bloques con los módulos de los que se compone el computador.

El código que modela la CPU completa en el módulo *Top* se presenta en la Figura TODO:. Este módulo cuenta con las siguientes señales de entrada/salida:

6.1. Proceso de despliegue. Ejemplo

6.1.1. Obtención del HDL de bajo nivel

Para generar código Verilog a partir de un desarrollo en Chisel [25] es necesario añadir la declaración de objeto mostrada en las líneas 18 a 20 de la Figura 6.2. En esa figura se muestra un pequeño ejemplo

de implementación en Chisel de un multiplexor de dos entradas (*Mux2*) y, seguido a esta, se declara un objeto *Mux2* que extiende la clase *App*.

Por defecto, un fichero de código desarrollado en Chisel no cuenta con un método principal que pueda ejecutarse, si no que simplemente consta de un conjunto de clases que definen el comportamiento interno de distintos componentes hardware. La instanciación de ese objeto *Mux2* extendiendo la clase *App* dota al código compilado de un punto de entrada en el que se ejecuta el método `emitVerilog`, cuyo resultado será el equivalente en sintaxis Verilog de la clase instanciada en el parámetro del método.

Chisel	Verilog
<pre> class Mux2 extends Module { val io = IO(new Bundle{ val i1 = Input(Bool()) val i2 = Input(Bool()) val sel = Input(Bool()) val o = Output(Bool()) }) io.o := MuxLookup(io.sel, 0.B)(Seq(0.U -> io.i1, 1.U -> io.i2)) } object Mux2 extends App { (new chisel3.stage.ChiselStage) .emitVerilog(new Mux2()) } </pre>	<pre> module Mux2(input clock, input reset, input io_i1, input io_i2, input io_sel, output io_o); assign io_o = io_sel ? io_i2 : io_i1; endmodule </pre>

Figura 6.2: Izda.: implementación de un multiplexor de dos entradas en Chisel y obtención de su equivalente en sintaxis Verilog. Dcha.: descripción del módulo en Verilog.

6.1.2. Configuración del fichero de *constraints*

Ya con la implementación en Verilog se puede pasar a crear un proyecto en Vivado con el que realizar, de manera automatizada, las transformaciones y procedimientos necesarios para generar el *bitstream*¹ que se cargará en la placa. Para mapear la entrada/salida del módulo sobre el hardware disponible en la placa es necesario crear un fichero de *constraints* en el que se indique, por cada puerto de entrada/salida en el módulo, el componente hardware al que se conectará.

Se muestran en la Figura TODO: los mapeos realizados para probar el diseño del multiplexor de dos entradas, asignando las dos entradas de datos y la señal de selección a tres interruptores deslizantes y, la salida del multiplexor, a uno de los LEDs disponibles.

¹El bitstream describe cómo debe configurarse el tejido programable de la FPGA, incluyendo la configuración de LUTs, registros, bloques de memoria, redes de interconexión y recursos de entrada/salida

```
1 set_property -dict { PACKAGE_PIN A10   IOSTANDARD LVCMOS33 } [get_ports { io_i1  }];  
2 set_property -dict { PACKAGE_PIN C10   IOSTANDARD LVCMOS33 } [get_ports { io_i2  }];  
3 set_property -dict { PACKAGE_PIN A8     IOSTANDARD LVCMOS33 } [get_ports { io_sel }];  
4 set_property -dict { PACKAGE_PIN H5     IOSTANDARD LVCMOS33 } [get_ports { io_o   }];
```

Figura 6.3: *Constraints* configurados para probar el multiplexor de dos entradas en la placa.

6.2. Modificaciones realizadas en el diseño de la CPU

TODO:

7 Conclusiones y trabajo futuro

El objetivo primero del proyecto era la implementación, en el lenguaje de descripción de hardware Chisel, de un núcleo RISC-V segmentado capaz de procesar el conjunto no privilegiado de instrucciones base definido en la especificación RV32I del estándar.

Este trabajo tuvo como punto de partida un diseño monociclo incompleto, el cual se terminó de desarrollar como parte de un proceso de aprendizaje inicial. Esta fase de aprendizaje implicó, en primer lugar, la asimilación de los conceptos fundamentales inherentes al desarrollo en lenguaje Chisel. Además, se realizó un notorio ejercicio de indagación para comprender el funcionamiento interno del núcleo, las características internas y el conexionado de los módulos que lo integran, así como la metodología seguida en el proceso de pruebas tanto de componentes individuales, como de la arquitectura global.

Tras esta fase inicial, comenzó el proceso de elaboración de la arquitectura segmentada, el cual ha tenido como resultado dos implementaciones distintas: una segmentada en dos fases (*fetch*-resto), y la implementación final en cinco fases; siendo ambas el resultado de una metodología de pruebas basada en la ejecución de tests que cargasen en la memoria de la CPU códigos con los que generar los escenarios de especial interés que el núcleo debería resolver por medio de los mecanismos hardware implementados.

El empleo de GTKWave para el análisis de las trazas VCD generadas por los tests fue determinante en el proceso de depuración y validación del núcleo. La visualización detallada del comportamiento de la arquitectura en cada ciclo permitió detectar dependencias entre instrucciones, comprender su origen y verificar el correcto funcionamiento de los mecanismos implementados para su resolución, todo ello de una manera ágil y visual.

Chisel ha demostrado ser el HDL intuitivo y eficaz que sus desarrolladores idearon. Su integración con el ecosistema de Scala y el empleo de conceptos propios de la programación orientada a objetos facilitan la construcción de componentes reutilizables y la organización modular del diseño, características que han permitido, sin duda, abordar la complejidad creciente de la arquitectura.

Como trabajo futuro podría plantearse la implementación de las instrucciones de manejo del CSR contempladas en la extensión Zicsr, lo cual sentaría las bases para la ejecución de un sistema operativo embebido como pudiera ser FreeRTOS.

Bibliografía

- [1] Krste Asanović y David A Patterson. «Instruction sets should be free: The case for risc-v». En: *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146* (2014). Consultado el 30 de agosto de 2025. URL: <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-146.pdf>.
- [2] OpenRISC Community. *Architecture*. Consultado el 28 de mayo de 2025. 2025. URL: <https://openrisc.io/architecture>.
- [3] Tony Chen y David A. Patterson. *RISC-V Geneology*. Inf. téc. UCB/EECS-2016-6. Accedido el 28 de mayo de 2025. University of California at Berkeley, 2016, pág. 2. URL: <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-6.pdf>.
- [4] RISC-V International. *RISC-V Landscape: Members*. Consultado el 28 de mayo de 2025. 2025. URL: <https://riscv.landscape2.io/?group=members>.
- [5] John Cocke y Victoria Markstein. «The evolution of RISC technology at IBM». En: *IBM Journal of research and development* 34.1 (1990). Consultado el 23 de agosto de 2025, págs. 4-11. URL: <https://ieeexplore.ieee.org/document/5389855>.
- [6] Andrew S Tanenbaum. «Implications of structured programming for machine architecture». En: *Communications of the ACM* 21.3 (1978). Consultado el 30 de agosto de 2025, págs. 237-246. URL: <https://dl.acm.org/doi/abs/10.1145/359361.359454>.
- [7] RISC-V International. *About RISC-V International*. Consultado el 30 de agosto de 2025. 2025. URL: <https://riscv.org/about/#history>.
- [8] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, pág. 9. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.
- [9] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, págs. 15-17. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.
- [10] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, págs. 27-35. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.
- [11] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, págs. 35-37. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.
- [12] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, págs. 37-38. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.

- [13] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, págs. 38-40. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.
- [14] Kito Cheng y Jessica Clarke. *RISC-V ABIs Specification*. Ver. August 12, 2025-draft. Consultado el 31 de agosto de 2025. RISC-V International. 2025, pág. 6. URL: <https://github.com/riscv-non-isa/riscv-elf-psabi-doc>.
- [15] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, págs. 25-26. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.
- [16] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, págs. 49-52. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.
- [17] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, págs. 47-48. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.
- [18] Jonathan Bachrach et al. «Chisel: constructing hardware in a scala embedded language». En: *Proceedings of the 49th annual design automation conference*. Consultado el 11 de julio de 2025. 2012, págs. 1216-1225. URL: <https://dl.acm.org/doi/abs/10.1145/2228360.2228584>.
- [19] VirtusLab. *Scala Survey Results 2023*. Consultado el 11 de julio de 2025. 2023. URL: <https://scalasurvey2023.virtuslab.com/>.
- [20] RISC-V International. *riscv-gnu-toolchain*. 2020. URL: <https://github.com/riscv-collab/riscv-gnu-toolchain>.
- [21] «IEEE Standard Hardware Description Language Based on the Verilog(R) Hardware Description Language». En: *IEEE Std 1364-1995* (1996). Consultado el 13 de julio de 2025, págs. 207-218. DOI: 10.1109/IEEESTD.1996.81542.
- [22] AMD. *7 Series FPGAs Data Sheet: Overview*. 2020. URL: https://docs.amd.com/v/u/en-US/ds180_7Series_Overview.
- [23] Scott Chacon y Ben Straub. *Pro Git*. 2.^a ed. Consultado el 15 de julio de 2025. Berkeley, CA: Apress, 2014. ISBN: 978-1-4842-0076-6. URL: <https://git-scm.com/book/en/v2>.
- [24] Stack Exchange Inc. *2022 Developer Survey*. Consultado el 15 de julio de 2025. 2022. URL: <https://survey.stackoverflow.co/2022/#technology-version-control>.
- [25] LF Projects LLC. *Frequently Asked Questions - Get me Verilog*. Consultado el 21 de junio de 2026. 2020. URL: <https://www.chisel-lang.org/docs/resources/faqs#get-me-verilog>.